Multi-modal understanding

Architecture 6: University teaching deck

How can a language model that reads tokens also see images, hear sound, and watch video?

Encoders for text, images, audio, and video, ImageBind, projection, query-based, and fusion connectors, and a multi-modal TinyAgent.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Multi-modal LLMs
Define a multi-modal LLM and name the components needed to build one: encoder, connector, LLM, and an optional generator. Explain why most models only encode inputs.

Encoders
Describe how images (ViT, CLIP), audio (wav2vec 2.0, Whisper, CLAP), and video (ViViT, TimeSformer, CLIP-based) become embeddings, and how ImageBind aligns many modalities.

Connectors
Compare projection-based (LLaVA), query-based (Q-Former), and fusion-based (Flamingo) connectors, and explain how to give TinyAgent image understanding.

Source: Ch. 9, PDF pp. 426–499

Why agents need more than text

Real-world tasks

Real applications are not limited to text. An agent may need to process a video, judge a website design from an image, or listen to your spoken request.

Multi-modal understanding

Understanding images, audio, and video, and not just written text, is multi-modal understanding. It is like watching a video and reading its caption, with the brain combining both signals.

The brain of the system

Multi-modal LLMs are often the main brain of single and multi-agent systems and are better at delegating tasks. Examples are GPT-5, Gemini 3.0, and the Claude 4.5 family.

What a multi-modal LLM is

More than text

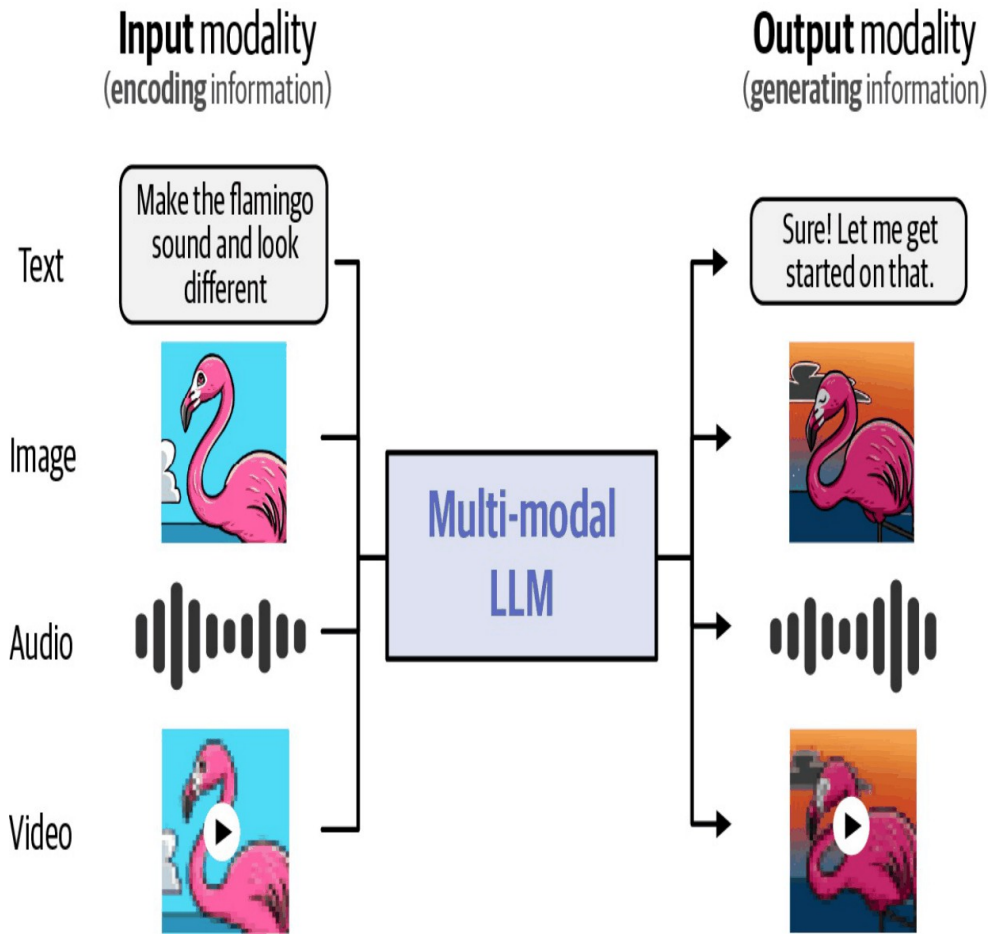
MLLMs are LLMs that can process and/or generate more types of data than text. Common modalities are text, image, audio, and video.

Input and output

A model may accept several input modalities, generate several output modalities, or both. The figure shows this range.

Fundamental change

Models that process and reason about images and audio fundamentally change how LLMs are used and how they interact with the world.



Grootendorst & Alammar (2026). Fig. 9-1, PDF p. 427

Most MLLMs encode inputs and generate text

Encoding is cheaper

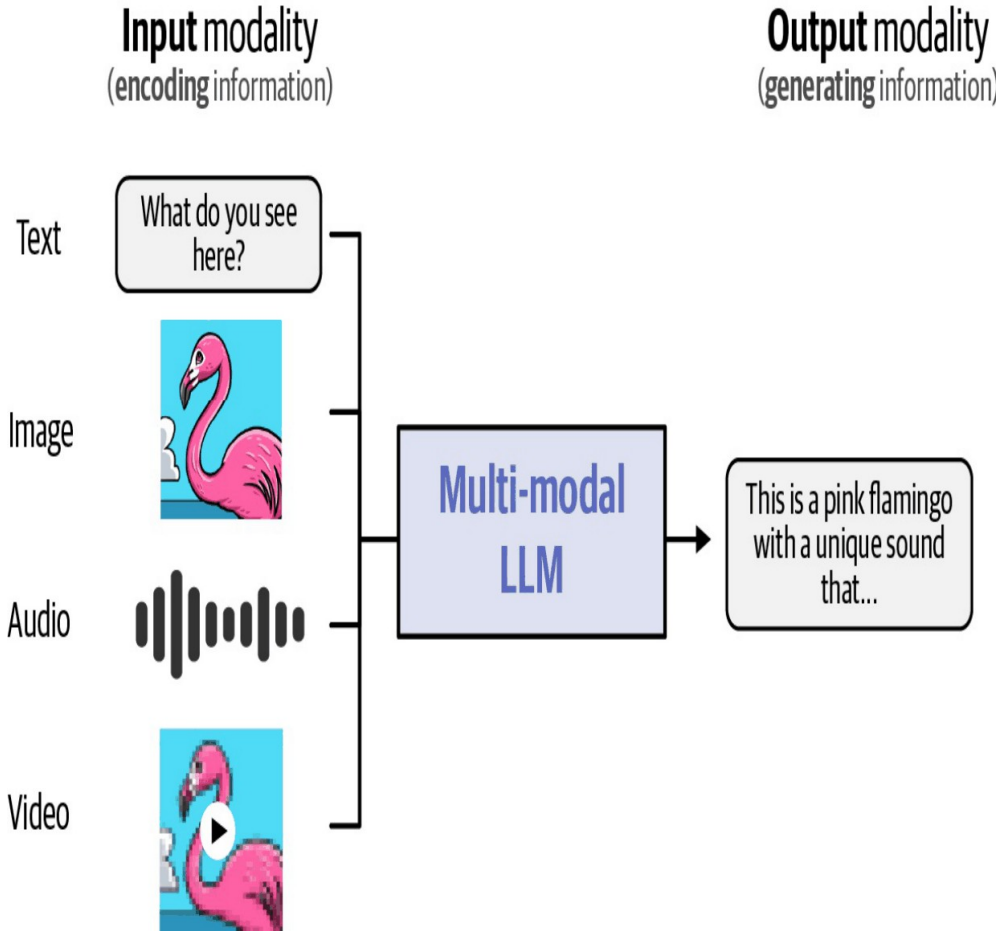
Most MLLMs focus on encoding input and not on generating other outputs. It is a much cheaper operation and needs no large architectural changes.

Context for text

Input modalities are often used to guide textual generation, like extra contextual information for a given task.

Single output

As the figure shows, the output is typically one modality: text.



Grootendorst & Alammar (2026). Fig. 9-2, PDF p. 428

An agent that can see the website

The example

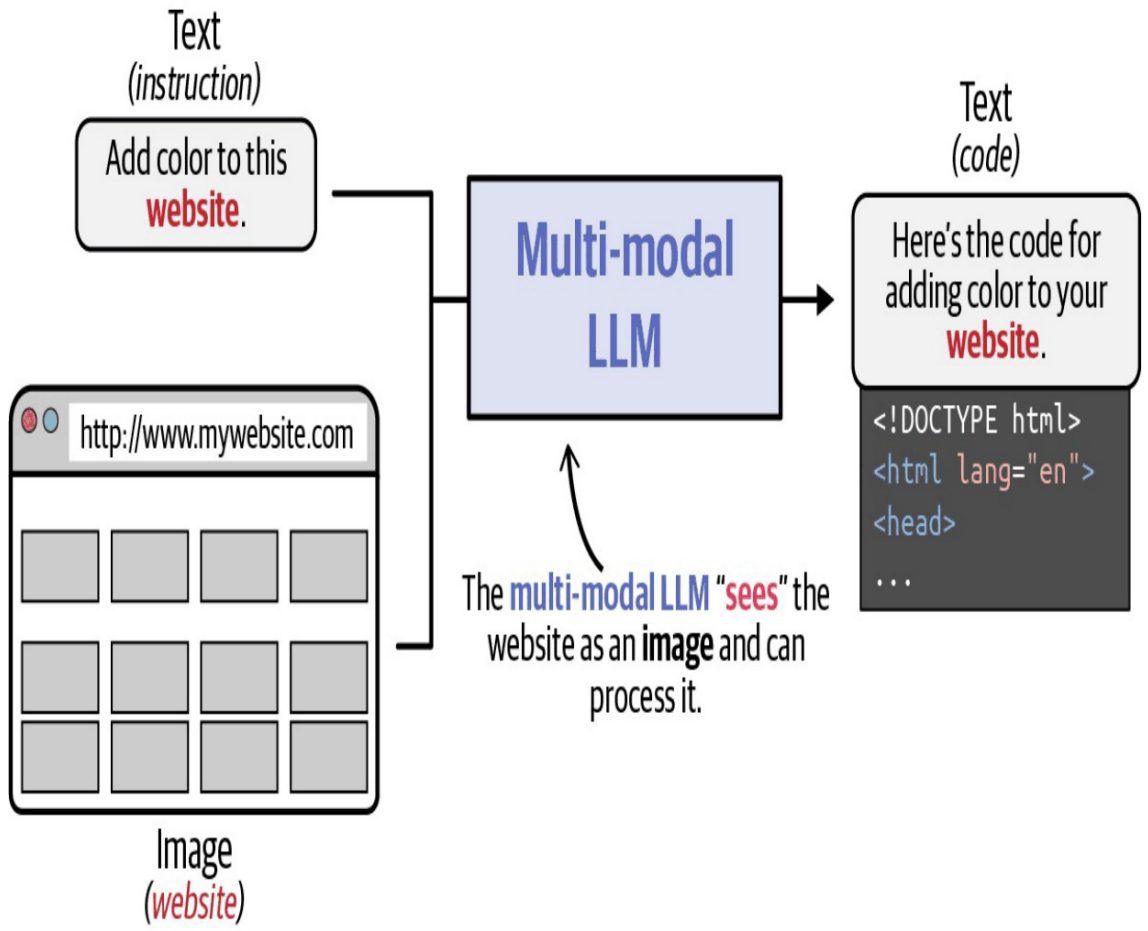
An LLM agent builds a website. It would help if the agent could actually see the website it has built.

Text plus image

The MLLM receives both the textual instruction and an image of the page, and uses both to optimize the design.

Why it matters

Without vision the agent must infer the visual result from code alone. With vision it can judge layout and appearance.



Grootendorst & Alammar (2026). Fig. 9-3, PDF p. 429

Three main components, one optional

Encoder and connector

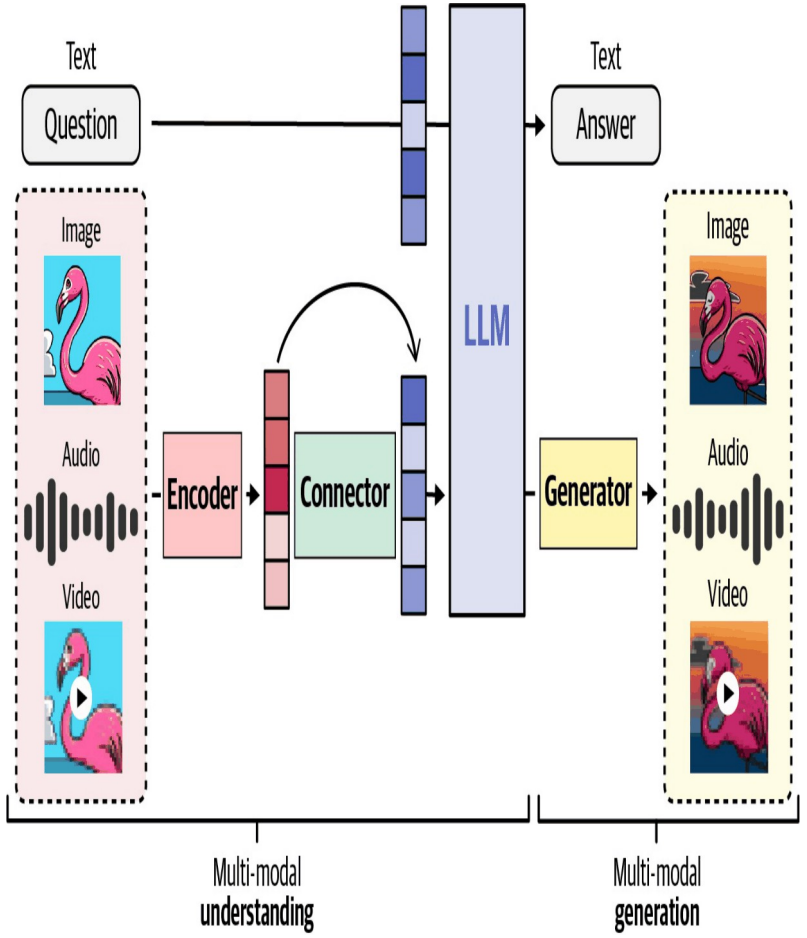
The encoder turns a modality such as an image or audio into features (embeddings). The connector converts those features so the LLM can use them.

LLM and generator

A pre-trained LLM processes the features and generates text. An optional generator produces modalities other than text.

Built in two stages

Creating an MLLM generally means training a base LLM separately and then adding multi-modality to it.



Grootendorst & Alammar (2026). Fig. 9-4, PDF p. 430

Scope: understanding, not generation

Input modalities

The chapter focuses on multi-modal understanding, meaning input modalities, and not on multi-modal generation.

Two questions

How can different modalities be encoded into features? And how can those features be converted so that the LLM can process them?

The roadmap

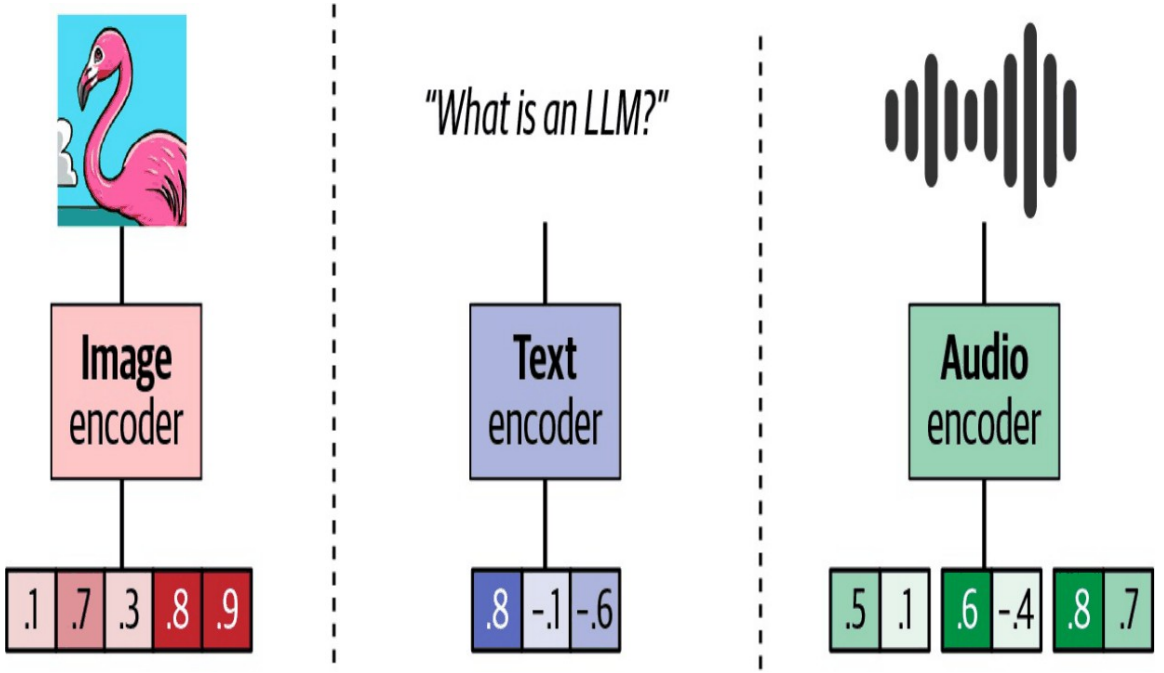
Encoders for text, images, audio, and video, then ImageBind for many modalities at once, then the three connector types, and finally a multi-modal TinyAgent.

Encoding turns input into embeddings

Common currency
Encoding a modality is generally converting an input, such as text, image, or audio, to embeddings.

One encoder per modality
Different modalities have different encoders and may produce different embeddings, as the figure shows for text, image, and audio.

Vital step
It is a vital part of making an LLM multi-modal, because the LLM expects embeddings as input.



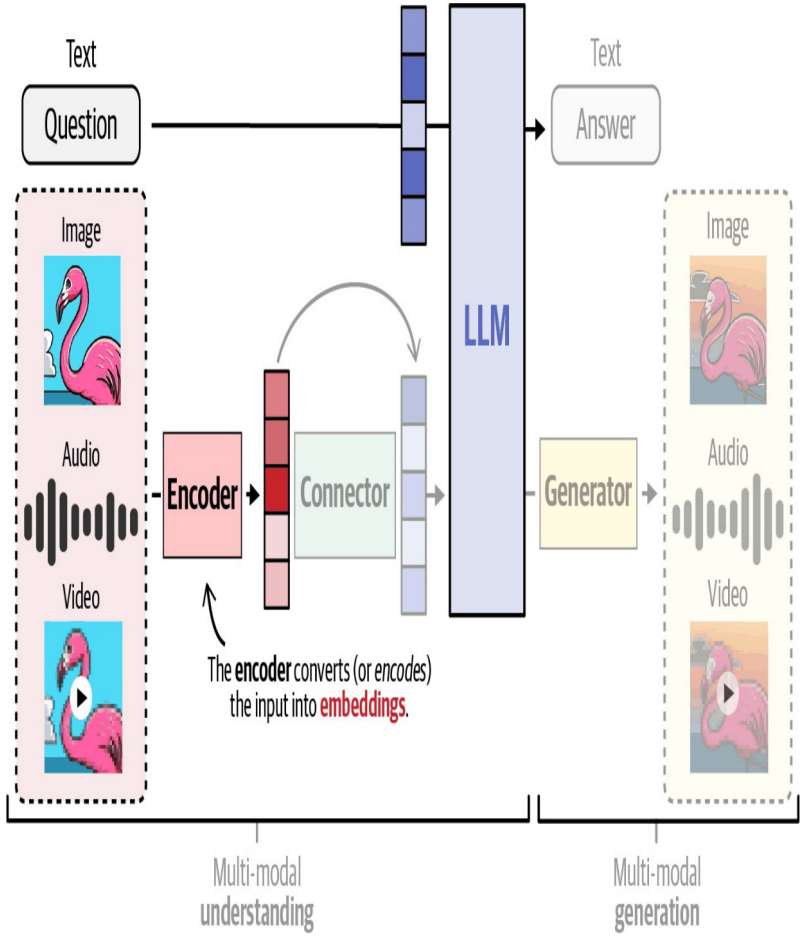
Grootendorst & Alammar (2026). Fig. 9-5, PDF p. 431

The encoder in the pipeline

Place of the encoder
In the multi-modal understanding pipeline, the encoder converts the input to embeddings before the LLM.

Start with the LLM
The chapter starts with the backbone, the LLM itself, to see why other modalities need different encoders.

Then the others
After text come images, audio, and video, each with a few representative methods.



Grootendorst & Alammar (2026). Fig. 9-6, PDF p. 432

How an LLM encodes text

Tokenizer

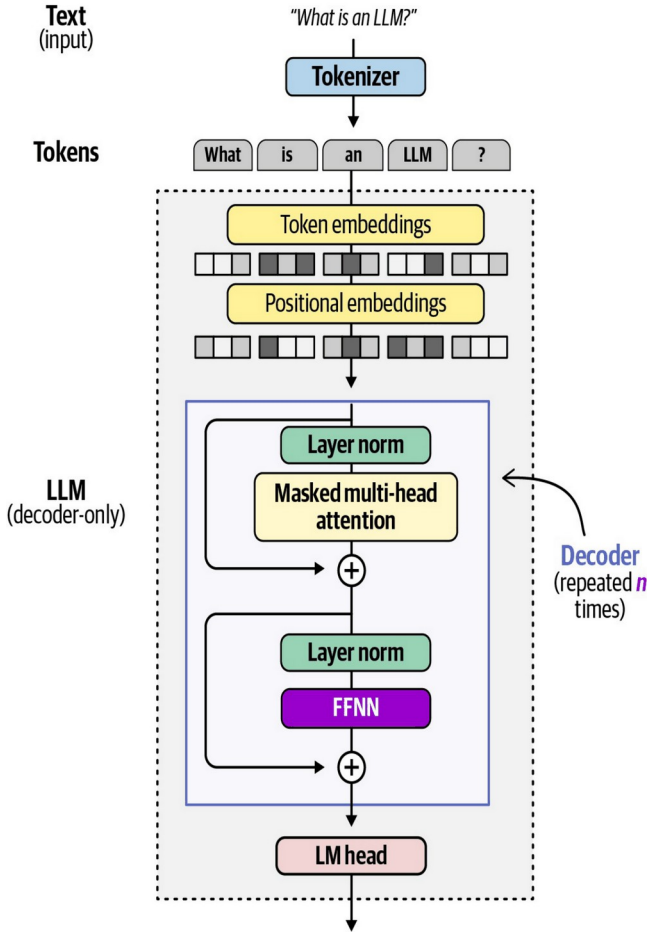
A tokenizer splits text into pieces (tokens). Each token is then embedded before it is passed to a decoder-only LLM.

Embedding layers

Token embeddings are combined with positional embeddings before the decoder blocks, which repeat n times.

The LM head

At the end, a language modeling head turns the result into a prediction for the next token.



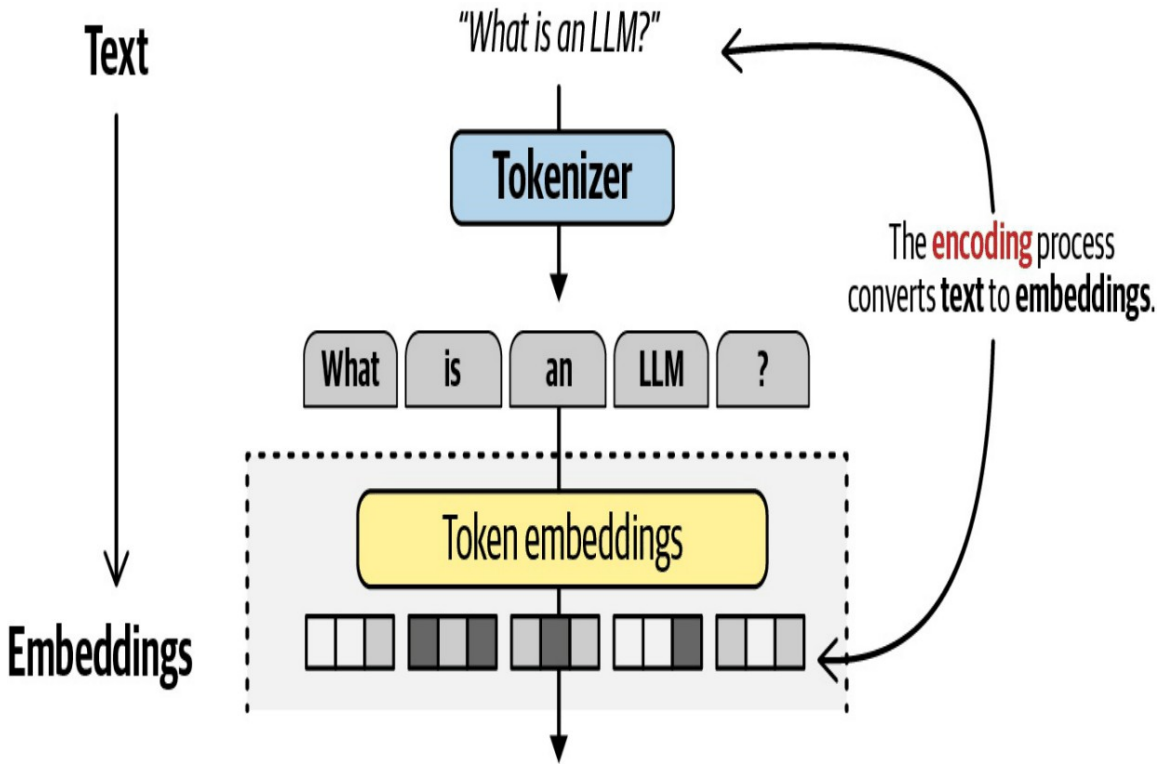
Grootendorst & Alammar (2026). Fig. 9-7, PDF p. 434

Text embeddings: tokenizer plus embedding layer

First part of the model
The first part of the Transformer converts text to embeddings: the tokenizer and the embedding layer.

The main input
Besides tokens and token IDs, the token embeddings are the main input that the LLM processes.

A place to add modalities
Embeddings are one of the primary places to add other modalities to an LLM.



Grootendorst & Alammar (2026). Fig. 9-8, PDF p. 435

Other modalities enter as embeddings

Same type of information

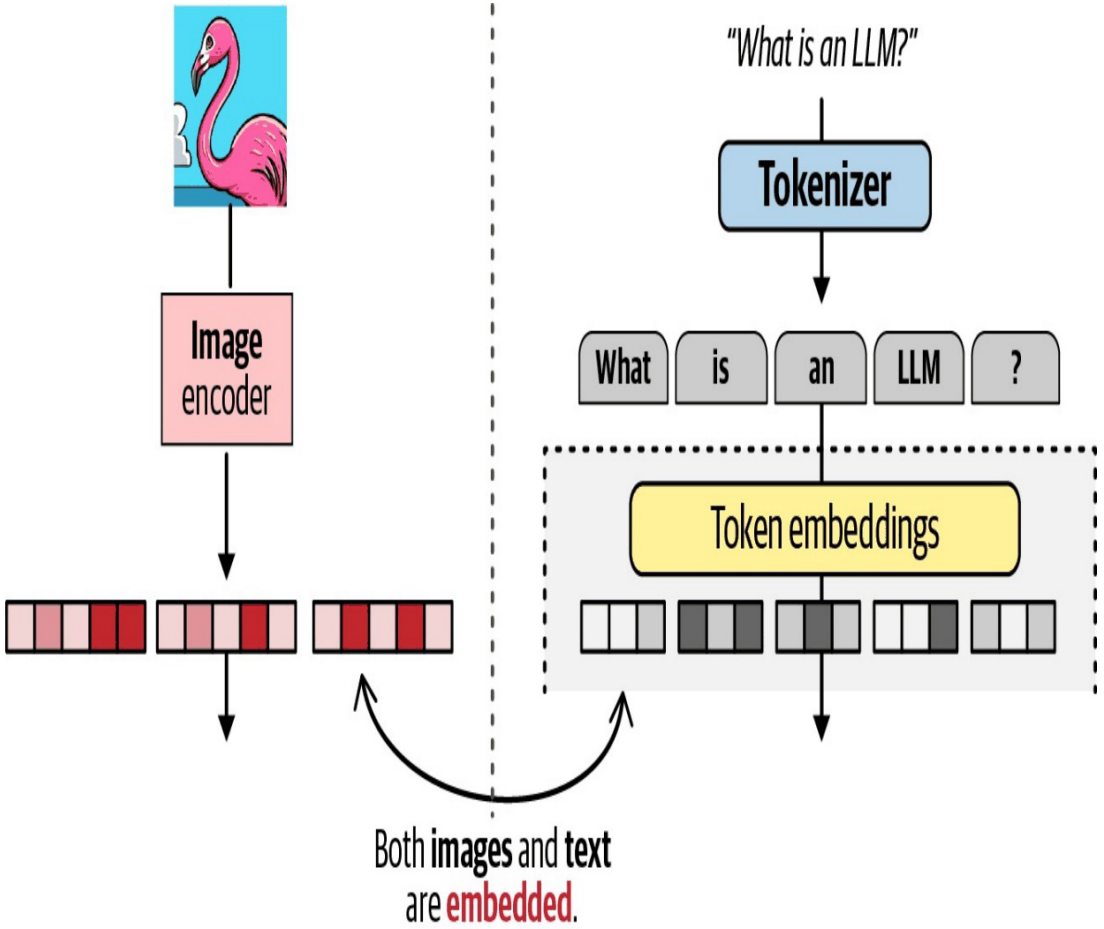
By encoding other modalities into embeddings, the LLM gets the same type of information to process, as the figure shows for an image encoder and the Transformer.

Two encoders

Both the image encoder and the Transformer convert their input into embeddings that can then be combined.

The key idea

A modality becomes usable for an LLM by being represented as embeddings, like tokens.



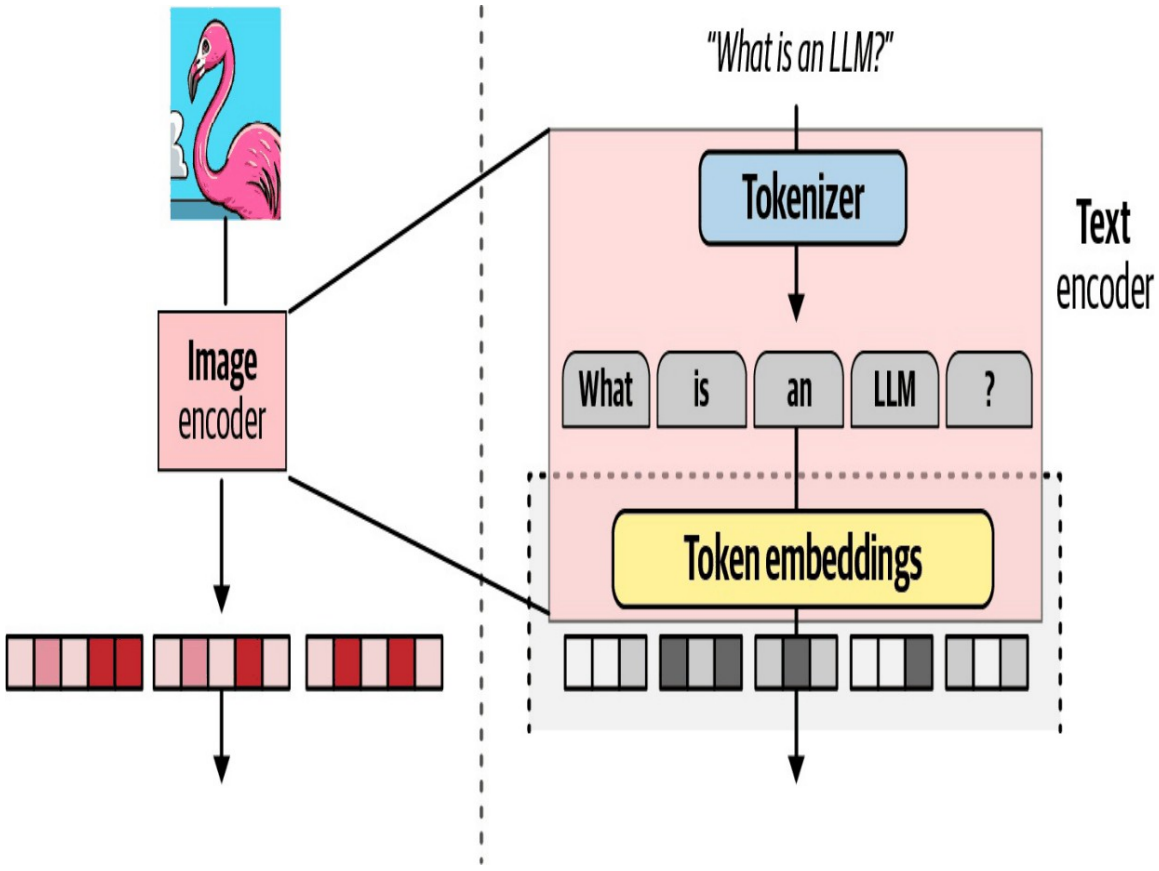
Grootendorst & Alammar (2026). Fig. 9-9, PDF p. 436

The text encoder is also an encoder

Reframing
The tokenizer together with the embedding layer (token embeddings) can be seen as the text encoder.

Symmetry
The Transformer converts text to embeddings in the same way an image or audio encoder would convert theirs.

A simple view
Each modality has an encoder. The LLM receives the resulting embeddings.



Grootendorst & Alammar (2026). Fig. 9-10, PDF p. 437

Embeddings from different encoders do not match

Different structure

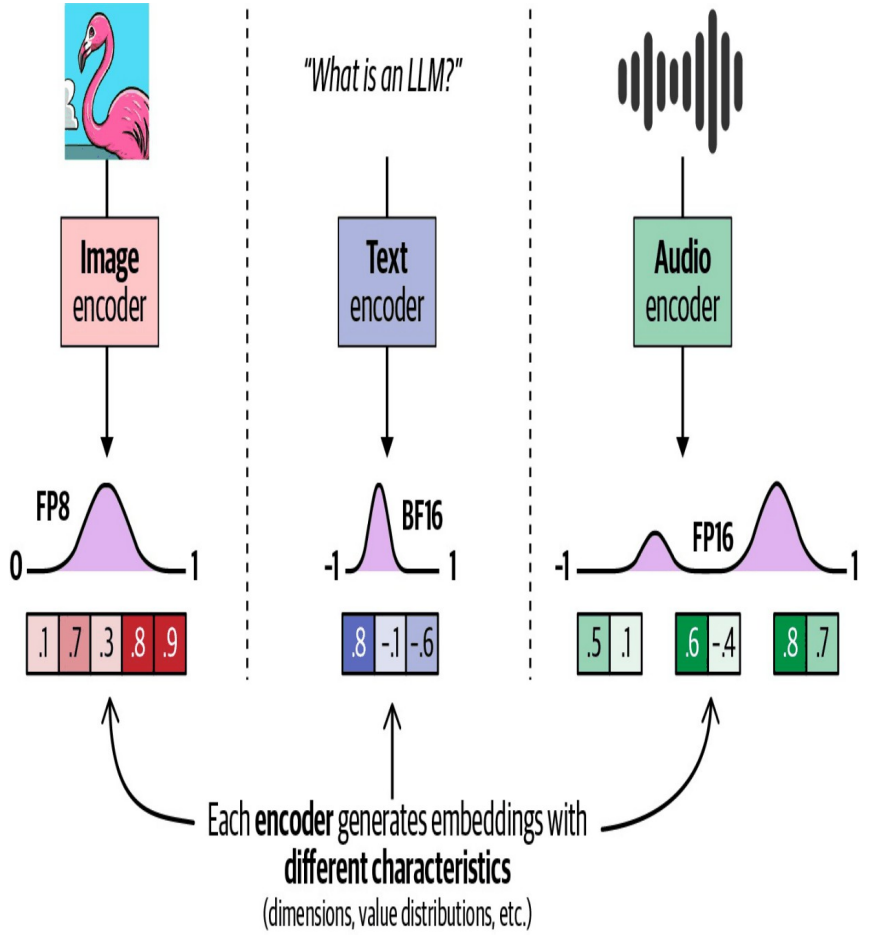
The structure and dimensions of embeddings from one modality, such as images, can be completely different from another, such as audio.

Trained on specific data

Each encoder is trained on specific data and generates different information, which cannot easily be compared.

A common projection

We need a model that projects embeddings onto common characteristics: the same dimensions and similar value distributions.



Grootendorst & Alammar (2026). Fig. 9-11, PDF p. 438

Vision Transformer: patches as words

An adapted Transformer

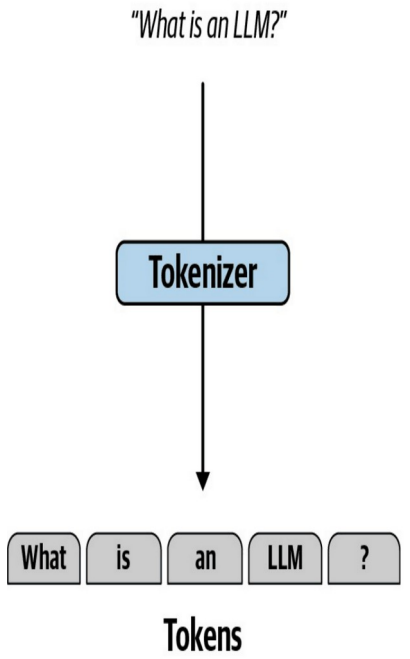
To encode images, an adaptation of the Transformer is used: the Vision Transformer (ViT).

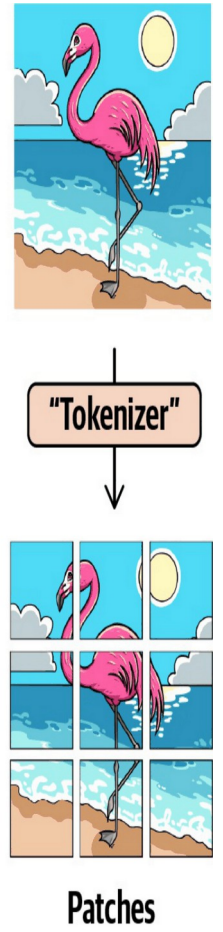
Mimic tokens

ViT creates patches of the image instead of tokens, and each patch is considered a word.

The first step

The image is cut into pieces, like a tokenizer cuts text. The figure shows the patches of a flamingo picture.





Grootendorst & Alammar (2026). Fig. 9-12, PDF p. 439

How ViT processes an image

Flatten and project

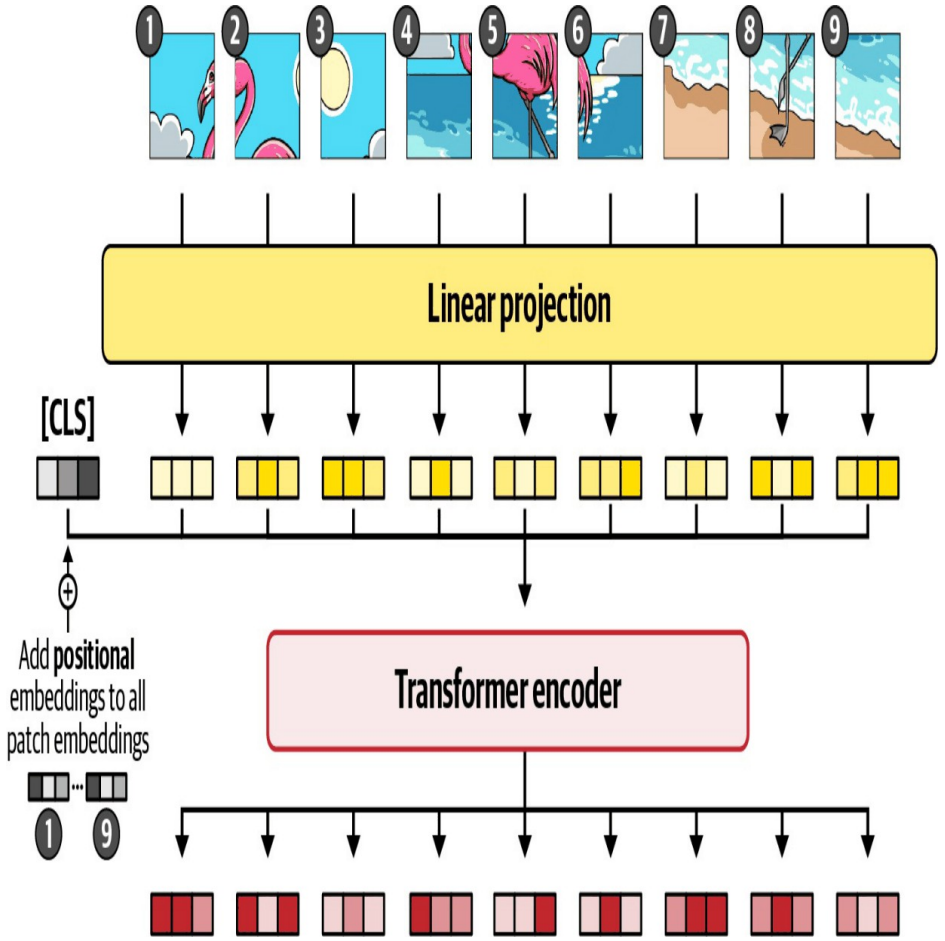
After the patching step, the patches are flattened and projected so that a regular Transformer encoder can process them as if they were tokens.

Contextual embeddings

The Transformer encoder returns embeddings for the patches that take the whole image into account.

Uses

The resulting embeddings can be used like regular embeddings, for classification, clustering, search, and more.



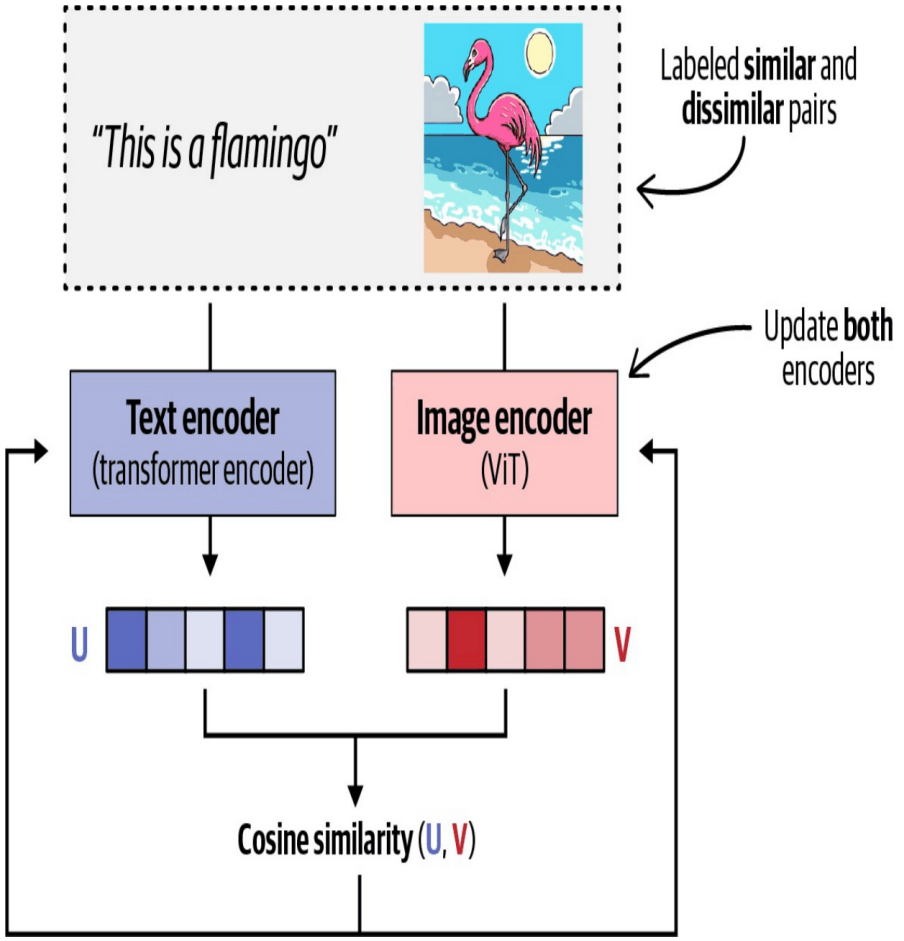
Grootendorst & Alammar (2026). Fig. 9-13, PDF p. 440

CLIP: contrastive language-image pre-training

Image and text
Arguably the most used image encoder encodes both. CLIP uses ViT for images and a Transformer encoder for text.

Contrastive learning
Using labeled similar and dissimilar image-text pairs, it aligns the embeddings of both modalities.

The result
Similar pairs get embeddings with high similarity. Dissimilar pairs get low similarity.



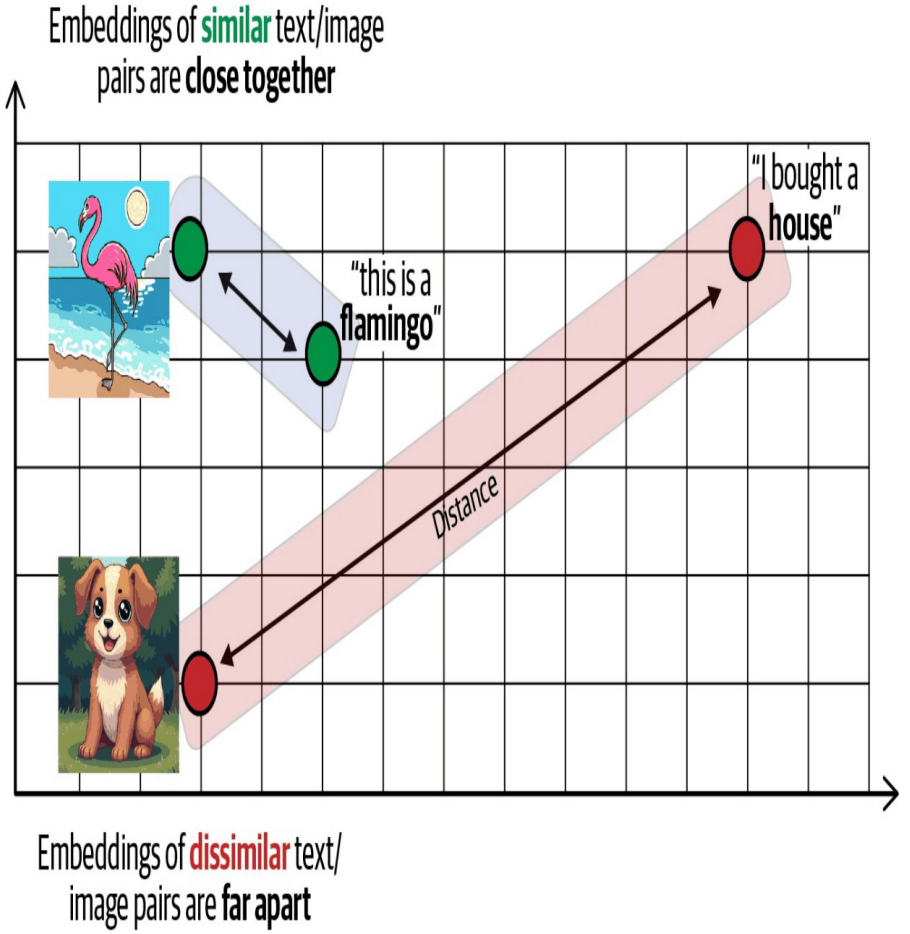
Grootendorst & Alammar (2026). Fig. 9-14, PDF p. 441

Semantic space and why CLIP is popular

Close means similar
Data points that are semantically similar lie close in the semantic space, and dissimilar ones are farther apart.

Powerful technique
By learning what is and is not similar, the model creates accurate semantic representations of the input.

Prior exposure to text
CLIP is a popular image encoder for MLLMs because its image embeddings have already had exposure to the textual modality.



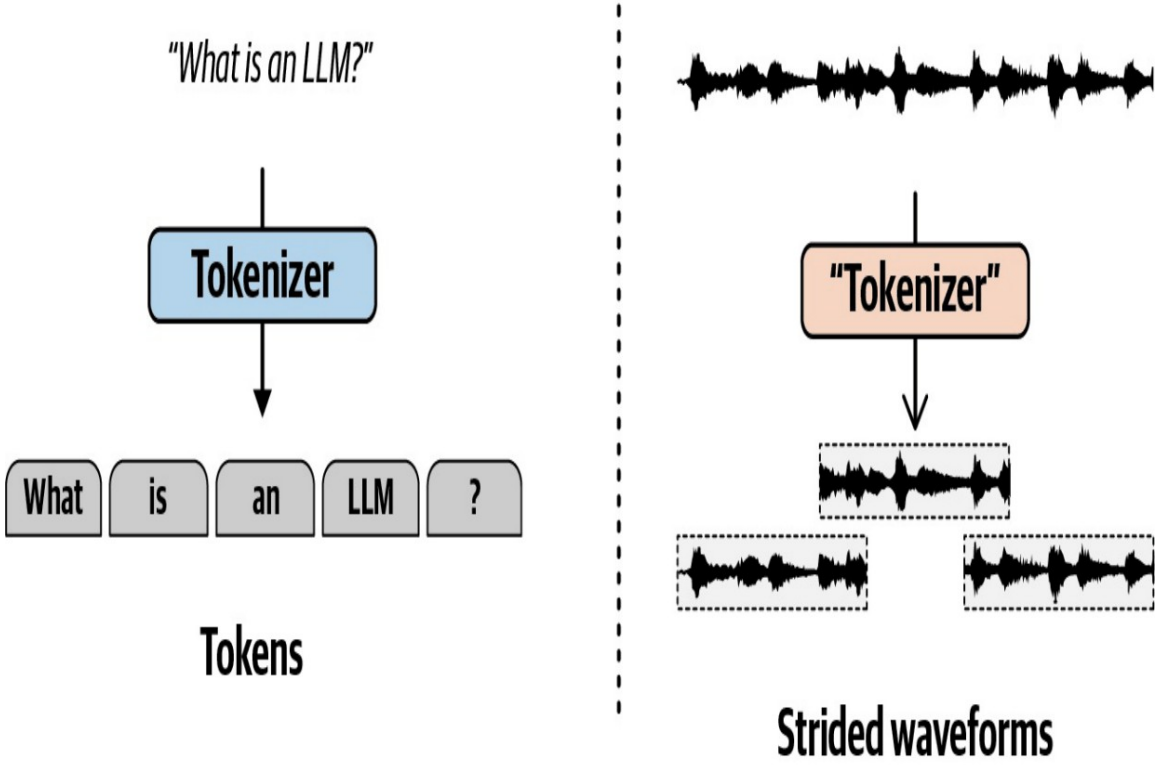
Grootendorst & Alammar (2026). Fig. 9-15, PDF p. 442

wav2vec 2.0: tokenizing sound

Audio encoders
Many methods for encoding audio use the Transformer and are partly inspired by the image and text encoders. One of the first successful ones is wav2vec 2.0.

Strided waveforms
It samples overlapping (strided) pieces of the audio to create tokens, like text and image tokenization.

Next step
Those pieces are processed by a convolutional neural network to create features.



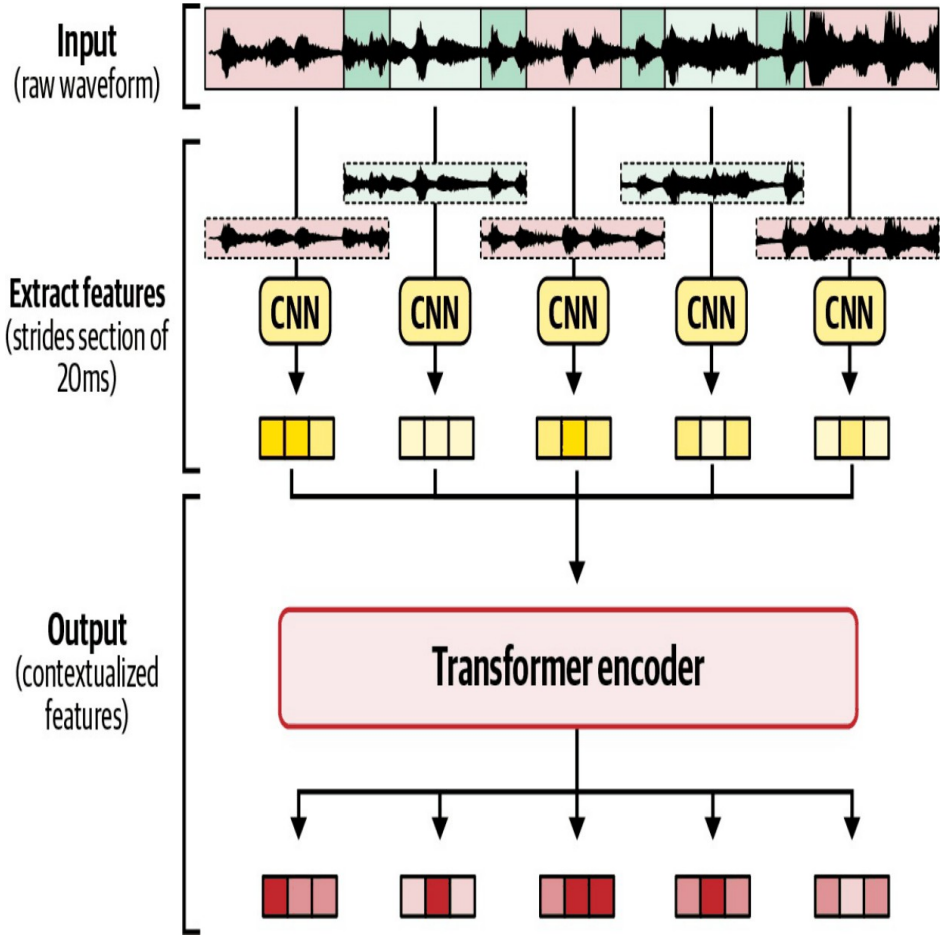
Grootendorst & Alammar (2026). Fig. 9-16, PDF p. 443

wav2vec 2.0: from waveform to contextual features

CNN features
A convolutional network turns each strided waveform piece into a feature vector (an embedding).

Transformer encoder
The features are passed to a regular Transformer encoder to create contextualized features.

Same pattern
Chop the input into pieces, embed the pieces, and let a Transformer add context, as for text and images.



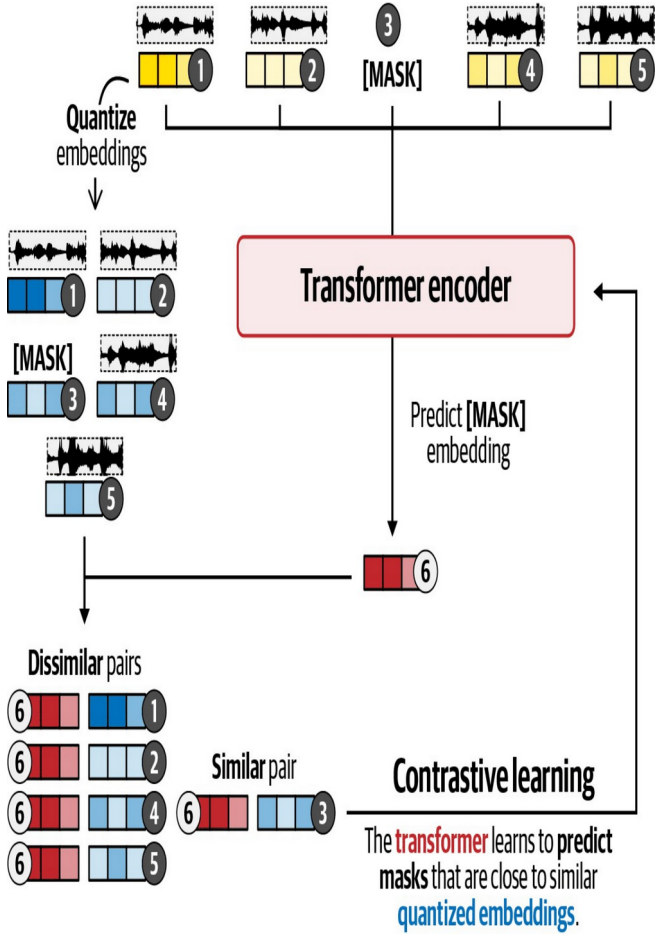
Grootendorst & Alammar (2026). Fig. 9-17, PDF p. 444

Training wav2vec 2.0

Masked modeling
Part of the input is randomly masked, and the Transformer must predict it.

Quantized embeddings
A codebook of quantized embeddings reduces the number of embeddings to represent. This works like a vocabulary.

Contrastive learning
The model is updated to produce embeddings close to similar inputs and far from dissimilar ones. HuBERT replaces quantization with clustering.



Grootendorst & Alammar (2026). Fig. 9-18, PDF p. 446

Whisper: a spectrogram and an encoder-decoder

Speech recognition

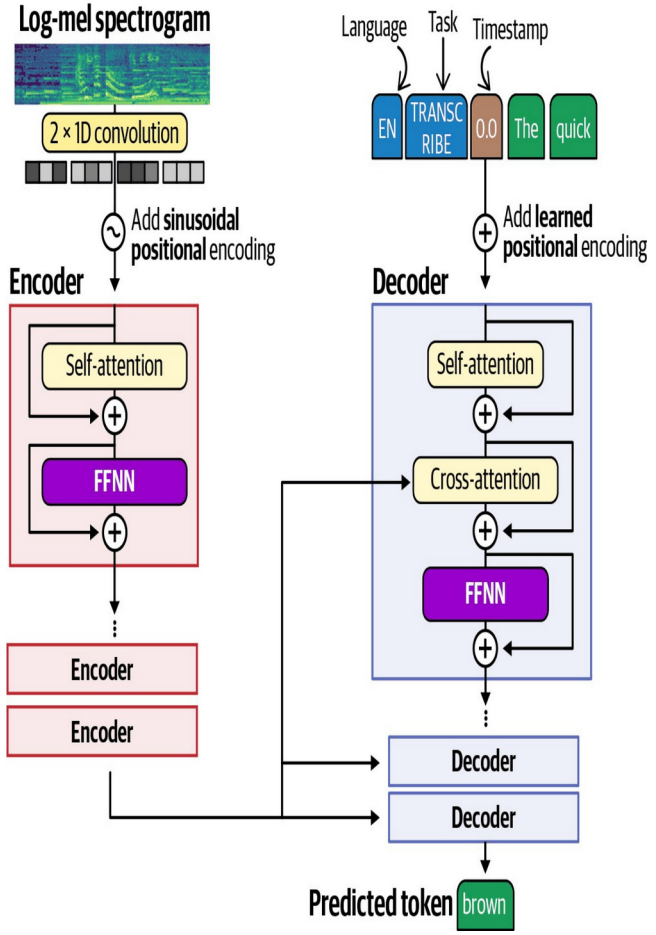
Whisper is used for automatic speech recognition. Its encoder represents the audio and its decoder generates the transcription.

Spectrogram

The audio is converted to a spectrogram, an image of sound showing how pitch and loudness change over time. Convolutional layers extract features.

Task tokens

Special tokens such as TRANSCRIBE or TRANSLATE tell the decoder which task to do. Others mark language, speech, and time.



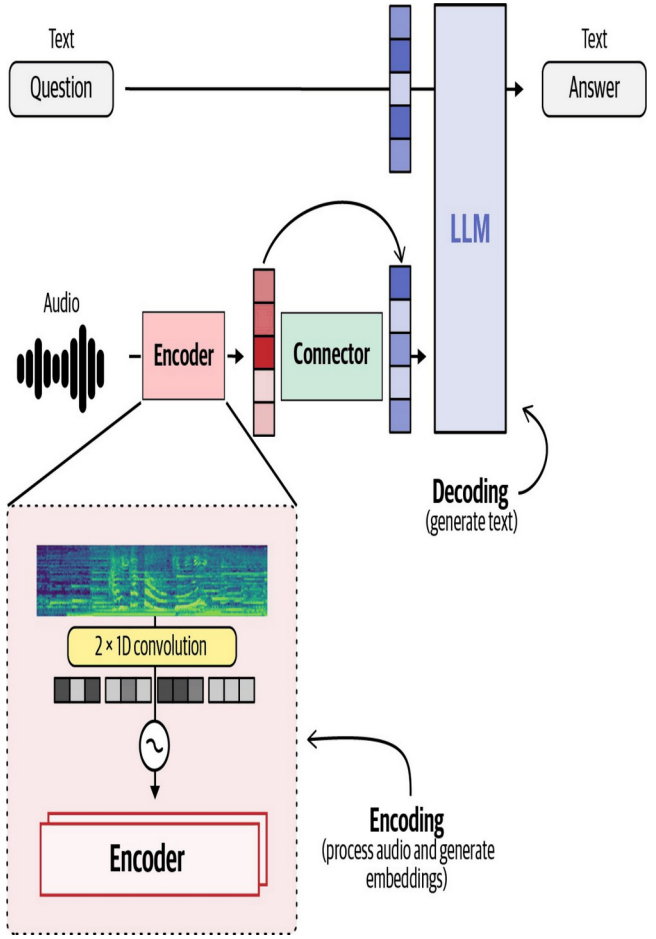
Grootendorst & Alammar (2026). Fig. 9-19, PDF p. 448

Whisper as the audio encoder of an MLLM

Encoder only
When Whisper is used in MLLMs, typically only the encoder is used, because it creates the audio representations.

The LLM generates
The LLM handles the generative part, so the Whisper decoder is not needed.

Prior text exposure
Like CLIP, Whisper is a popular encoder because its audio embeddings have already had exposure to the textual modality.



Grootendorst & Alammar (2026). Fig. 9-20, PDF p. 450

CLAP: contrastive language-audio pre-training

CLIP for audio

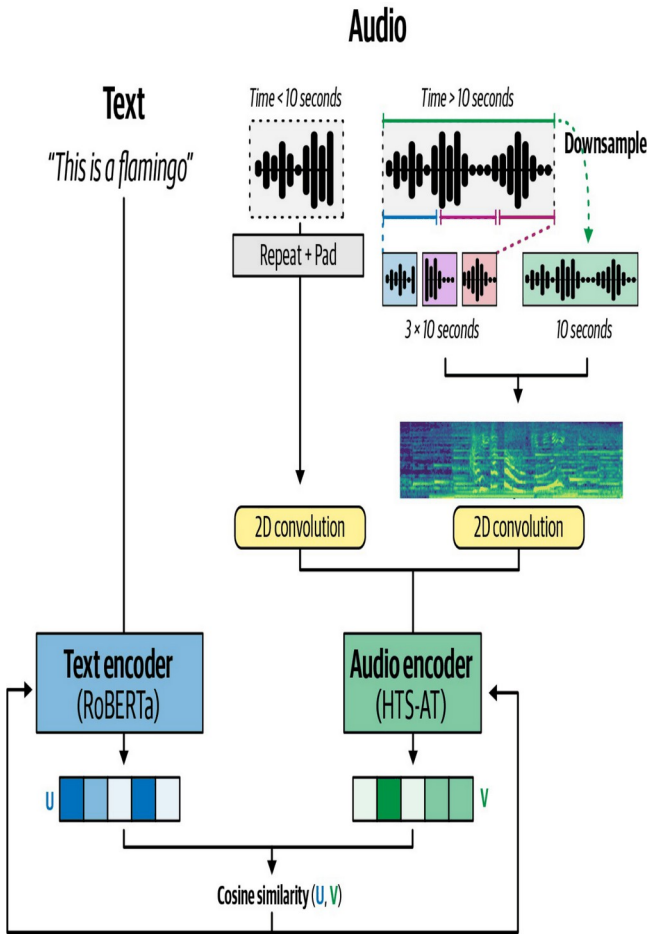
CLAP is based on CLIP but takes audio instead of images, trained contrastively on audio and text pairs, often captions of audio samples.

Fixed chunk length

For a chunk length such as 10 seconds: shorter audio is repeated and zero-padded. Longer audio is downsampled, and three random 10-second slices are taken.

Text and audio encoders

The figure shows a text encoder and an audio encoder, aligned by cosine similarity between their outputs.



Grootendorst & Alammar (2026). Fig. 9-21, PDF p. 452

HTS-AT: patches of a spectrogram

Best audio encoder

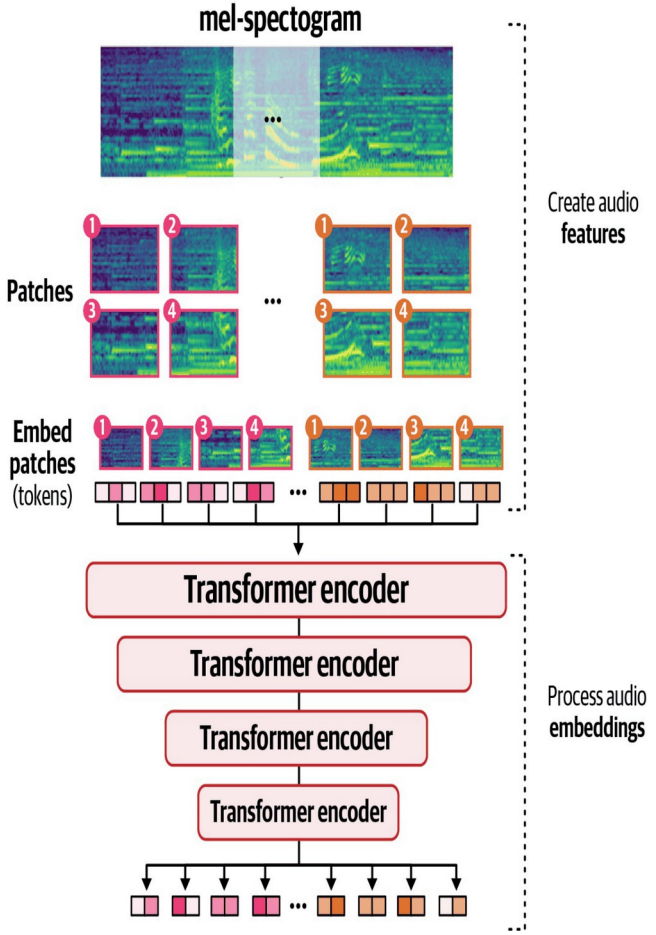
The authors of CLAP tested several audio encoders. The Hierarchical Token Semantic-Audio Transformer (HTS-AT) worked best.

Spectrogram patches

It converts the audio to a spectrogram and extracts patches to produce tokens, as in the image examples.

Hierarchical

Several hierarchical Transformer models downscale the large token embeddings.



Grootendorst & Alammar (2026). Fig. 9-22, PDF p. 454

Video adds a time dimension

Overlap with images

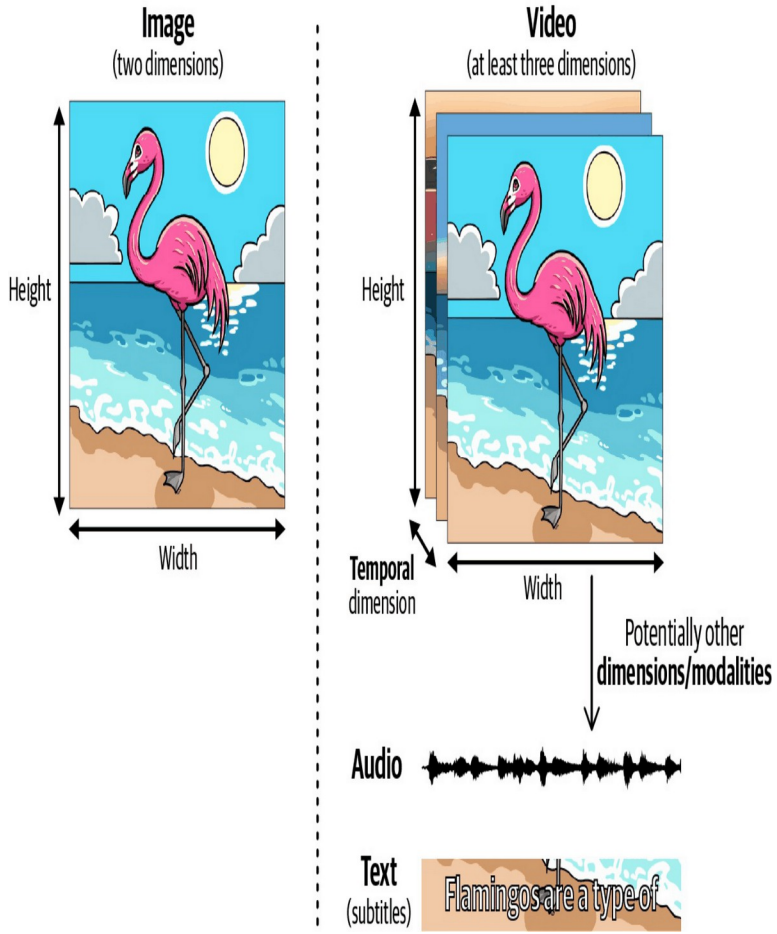
Encoding video overlaps with encoding images, but adds a temporal dimension encoded as a sequence of images.

More than frames

Audio and subtitles are tightly related and can be added to the encoding.

The challenge

The main challenge is modeling the temporal dimension. How do we keep track of a sequence of images?



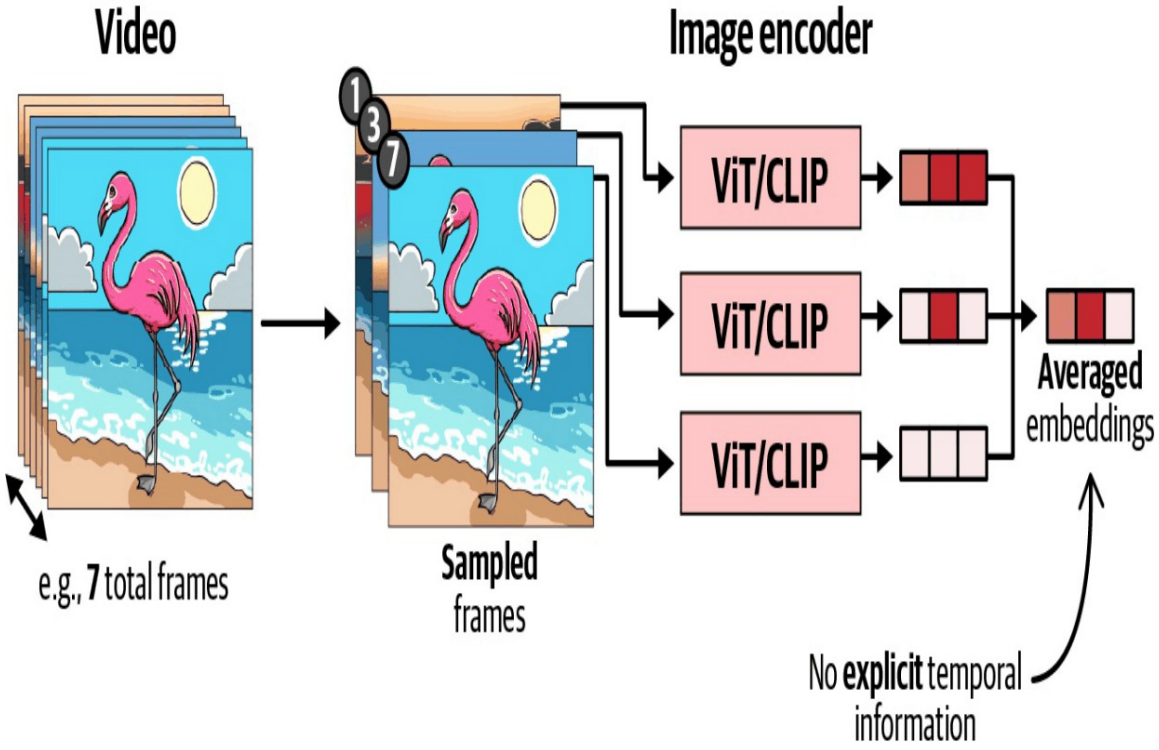
Grootendorst & Alammar (2026). Fig. 9-23, PDF p. 456

The basic approach: sample frames and average

Simple recipe
Use a ViT or CLIP-like model, sample a number of frames, embed them, and average the embeddings.

What is lost
Averaging removes the temporal dimension and prevents the LLM from understanding how the video started and ended.

The question
Are there more sophisticated ways to use ViT or CLIP embeddings? The following methods add temporal structure.



Grootendorst & Alammar (2026). Fig. 9-24, PDF p. 457

ViViT: uniform frame sampling

Spatial and temporal

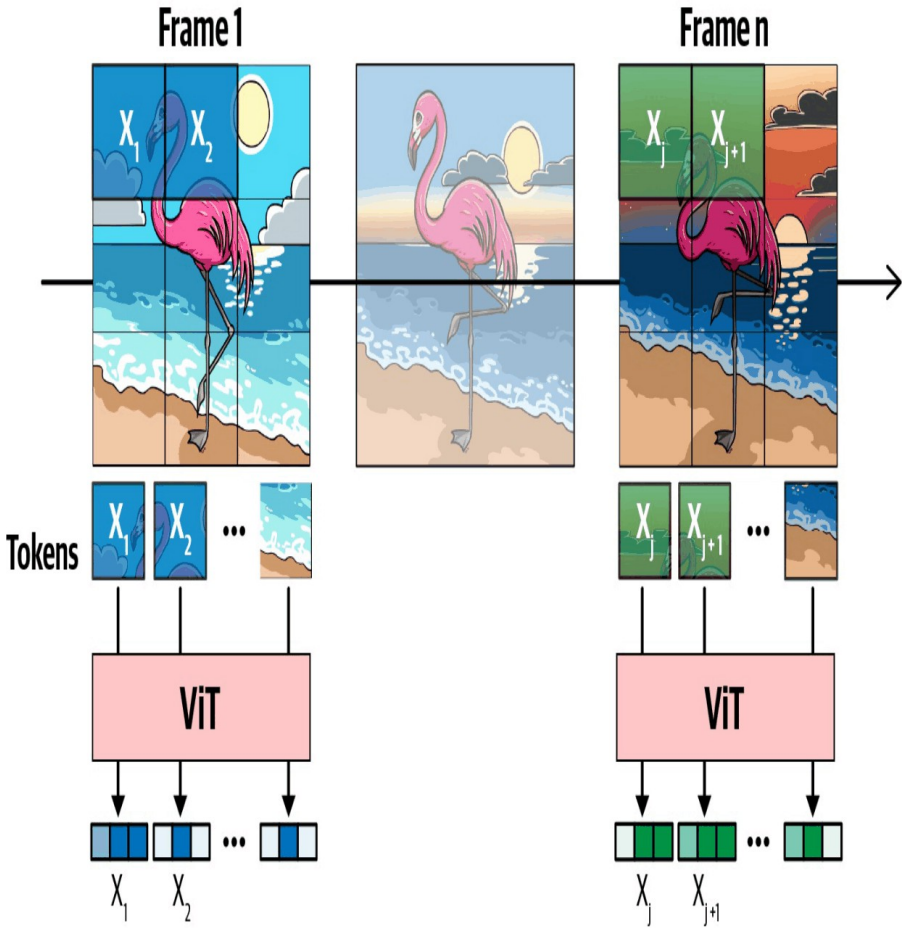
The Video Vision Transformer (ViViT) uses different combinations of Transformers for spatial information (the image itself) and temporal information (changes between frames).

First option

Uniform frame sampling: sample frames and pass them through ViT to create tokens for each patch of each frame.

Tokens per frame

Each sampled frame contributes its own set of patch tokens.



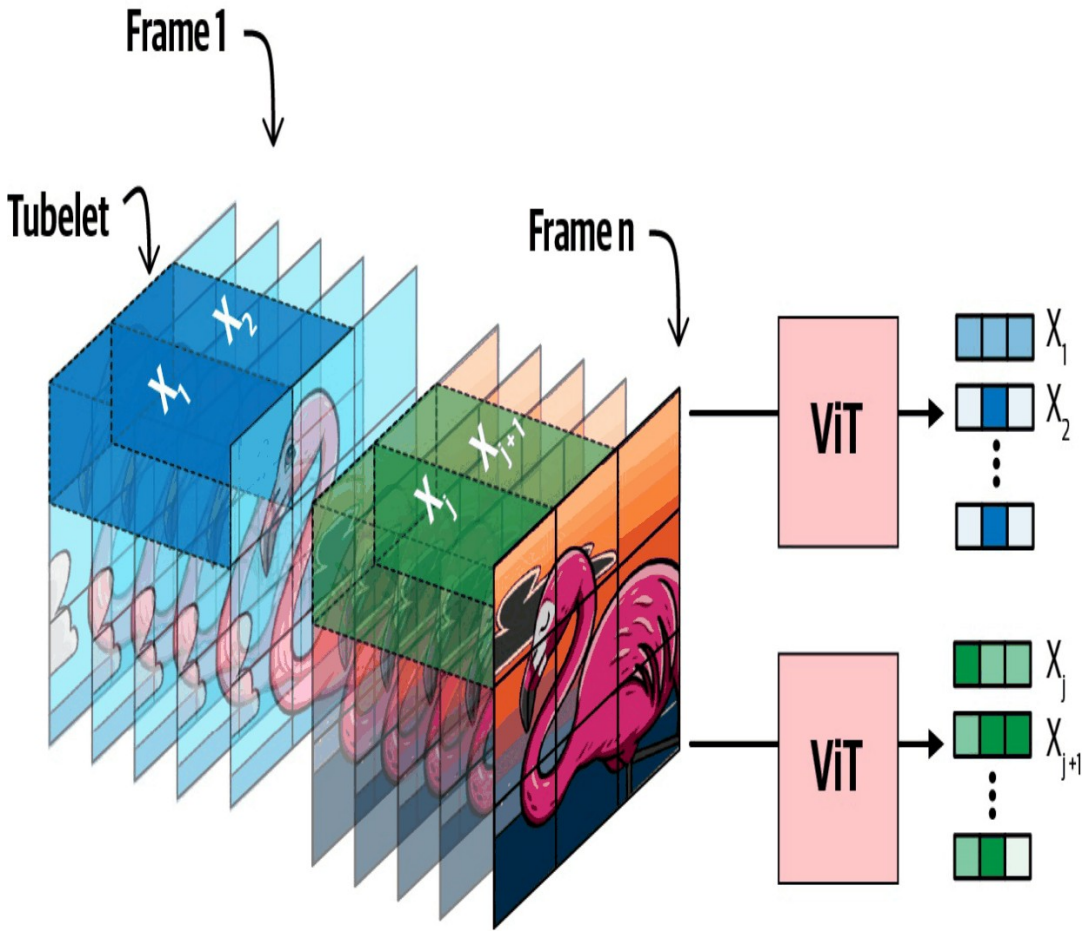
Grootendorst & Alammar (2026). Fig. 9-25, PDF p. 458

ViViT: tubelet embeddings

A 3D patch
A tubelet is a block of pixels from consecutive frames. It captures part of the image and how it changes over time.

Non-overlapping tubelets
Instead of a single image, non-overlapping tubelets spanning several frames are passed in, producing tubelet embeddings.

Space and time
A tubelet embedding spans both the spatial and the temporal nature of the video.



Grootendorst & Alammar (2026). Fig. 9-26, PDF p. 459

ViViT: encoders for space and time

Many designs

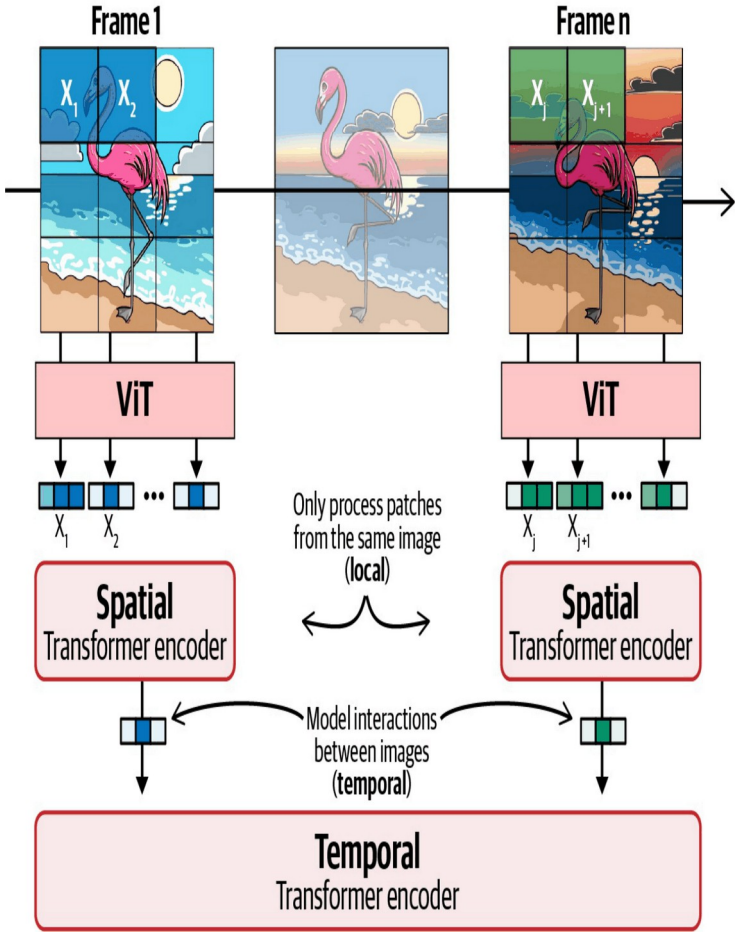
The authors experimented with several models for passing either type of embedding to Transformer encoders.

Spatial then temporal

The most prominent one passes the embeddings to a set of Transformer encoders that process mainly spatial information, followed by ones for temporal information.

Tokens everywhere

Every modality attempts to represent its input as some sort of token.



Grootendorst & Alammar (2026). Fig. 9-27, PDF p. 460

TimeSformer: patches into one Transformer

Separate dimensions

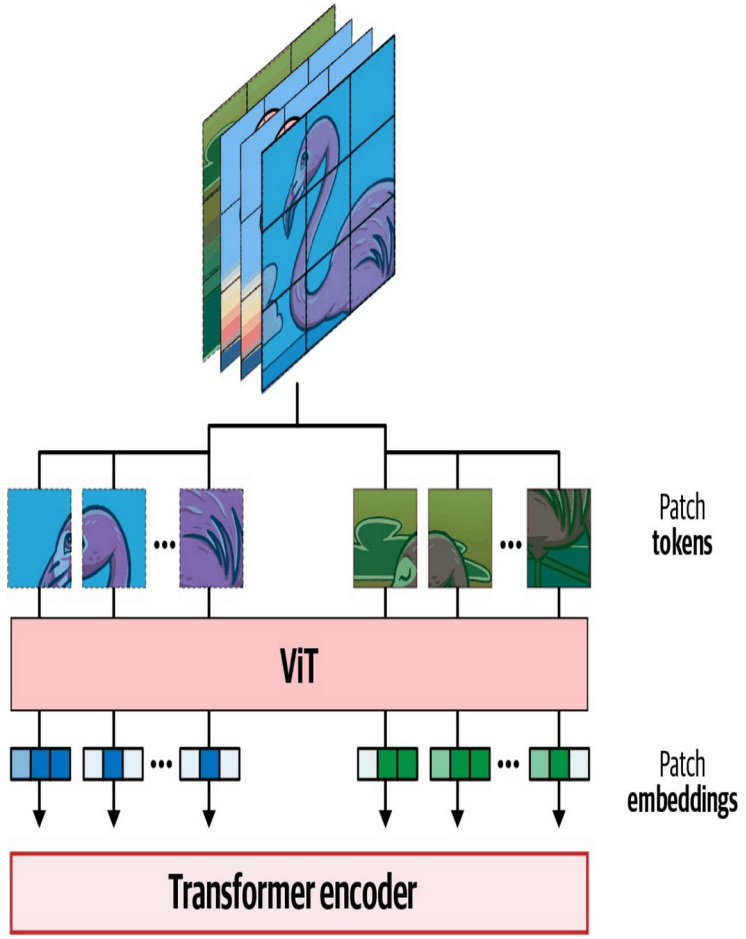
TimeSformer also separates temporal and spatial dimensions, but within a single Transformer model.

Frames to patches

As in ViViT, frames are sampled and split into patches. The patches are flattened and passed directly to a Transformer, not treated as tubelets.

Next

The difference is in the attention: it is divided into time attention and space attention.



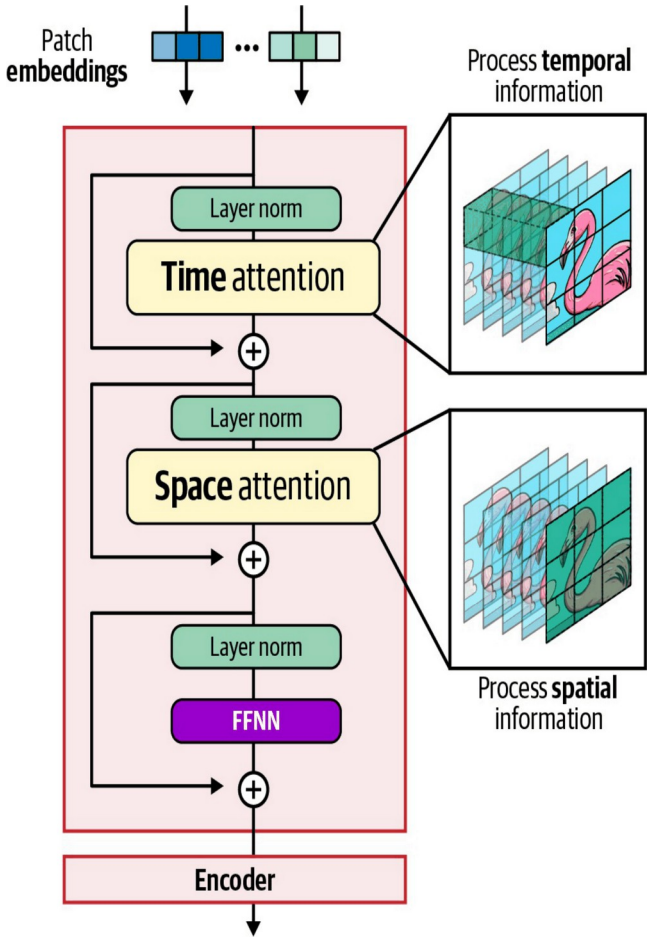
Grootendorst & Alammar (2026). Fig. 9-28, PDF p. 461

TimeSformer: divided space-time attention

Two attentions
Instead of different Transformers for time and space, it applies attention mechanisms that focus on different aspects: time attention and space attention.

Time attention
Uses tubelet-like groups along the time dimension to capture motion patterns.

Space attention
Works on patches within one frame to capture spatial structures such as object relationships.



Grootendorst & Alammar (2026). Fig. 9-29, PDF p. 463

Why divided attention helps

Better representations

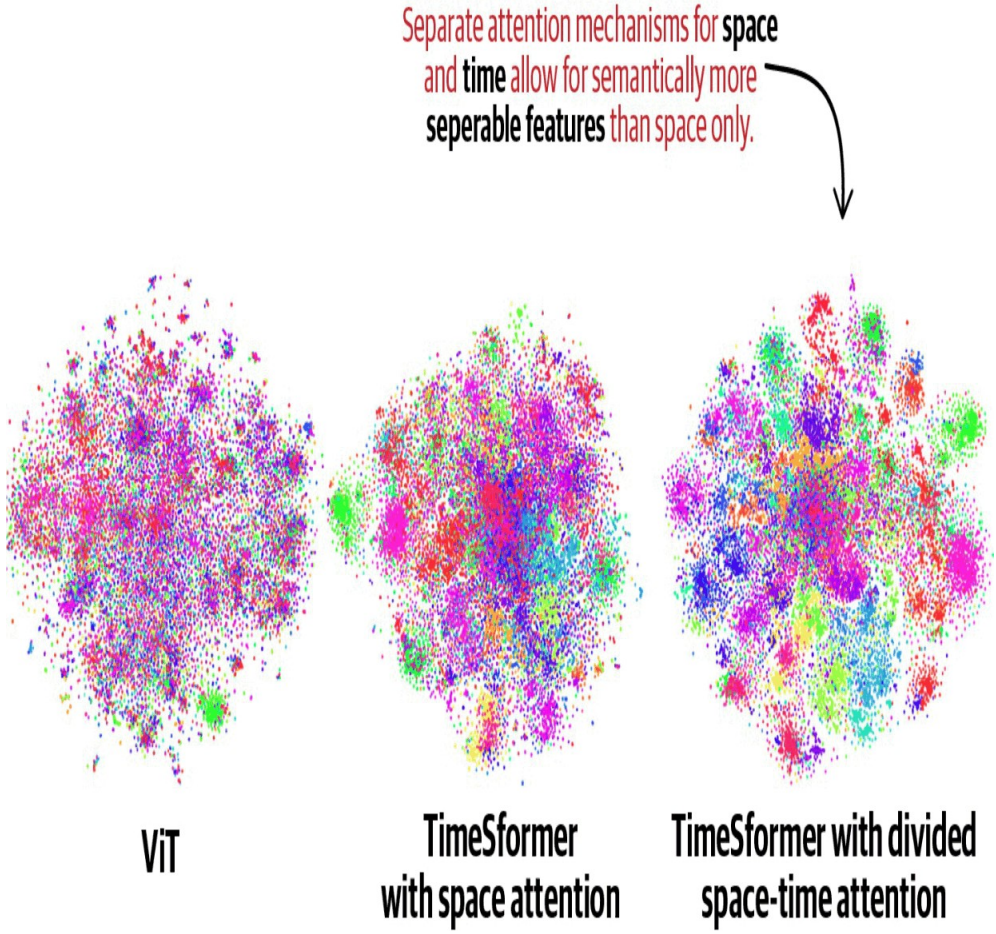
Divided space-time attention allows better representations than if only space attention were used, or than ViT alone.

An annotated figure

The figure is annotated from the paper Is Space-Time Attention All You Need for Video Understanding?

Shared principle

Sampling frames, creating patch tokens, embedding them, and using them in some variant of a Transformer encoder. VideoBERT, Frozen in Time, and VideoMAE do likewise.



Grootendorst & Alammar (2026). Fig. 9-30, PDF p. 464

"Just" CLIP: let the connector do the work

The pipeline

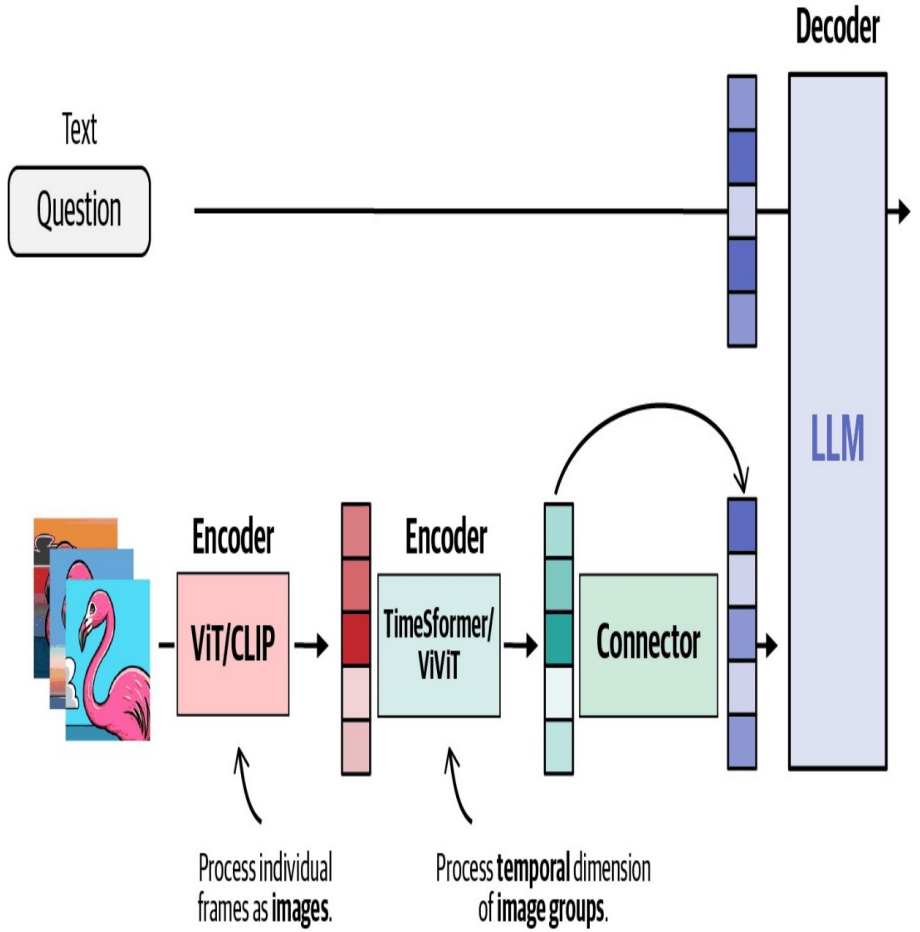
In the earlier methods the ViT or CLIP embeddings get extra temporal processing and are then passed to the LLM through a connector.

Redundancy

There are now two Transformer encoders before the LLM: the image encoder and the temporal encoder.

A simpler idea

Skip the second encoder and have the LLM figure out the temporal dimension.



Grootendorst & Alammar (2026). Fig. 9-31, PDF p. 465

Skipping the explicit temporal encoder

The connector does more

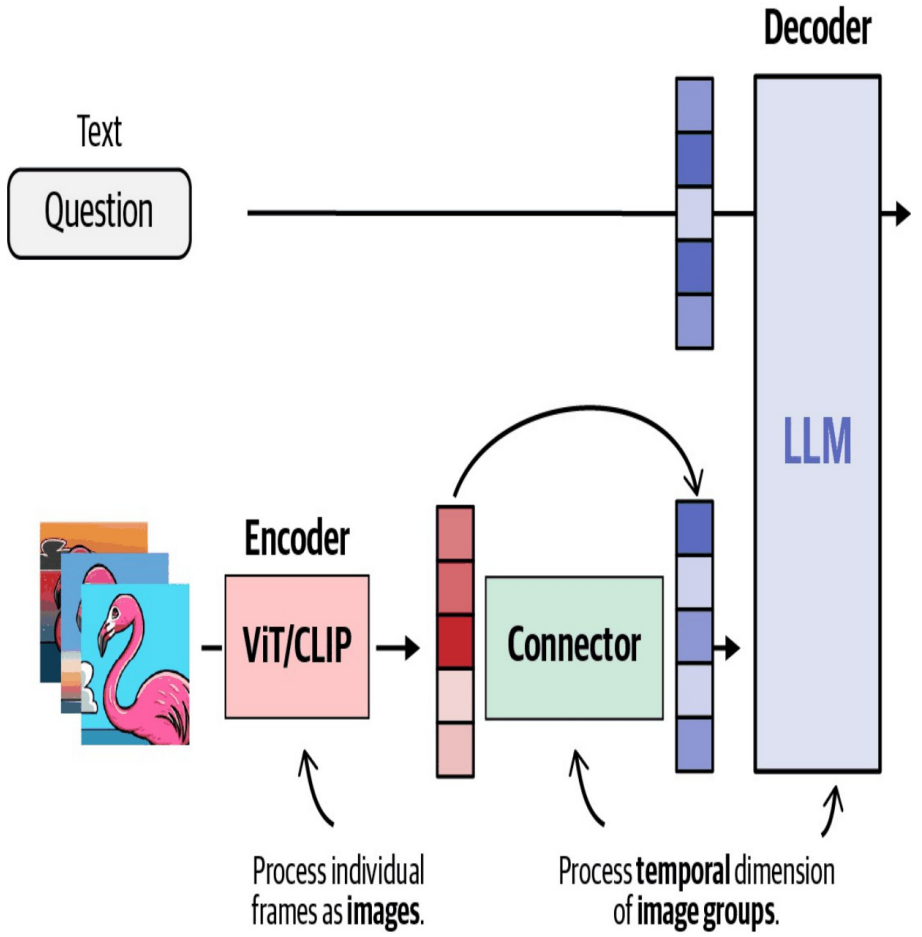
The connector can do a bit more processing, so no separate temporal encoder is needed.

LLMs know sequence

Decoder LLMs are already good at the temporal dimension of text. The same can work for video if each frame or patch is treated as a sequence of tokens.

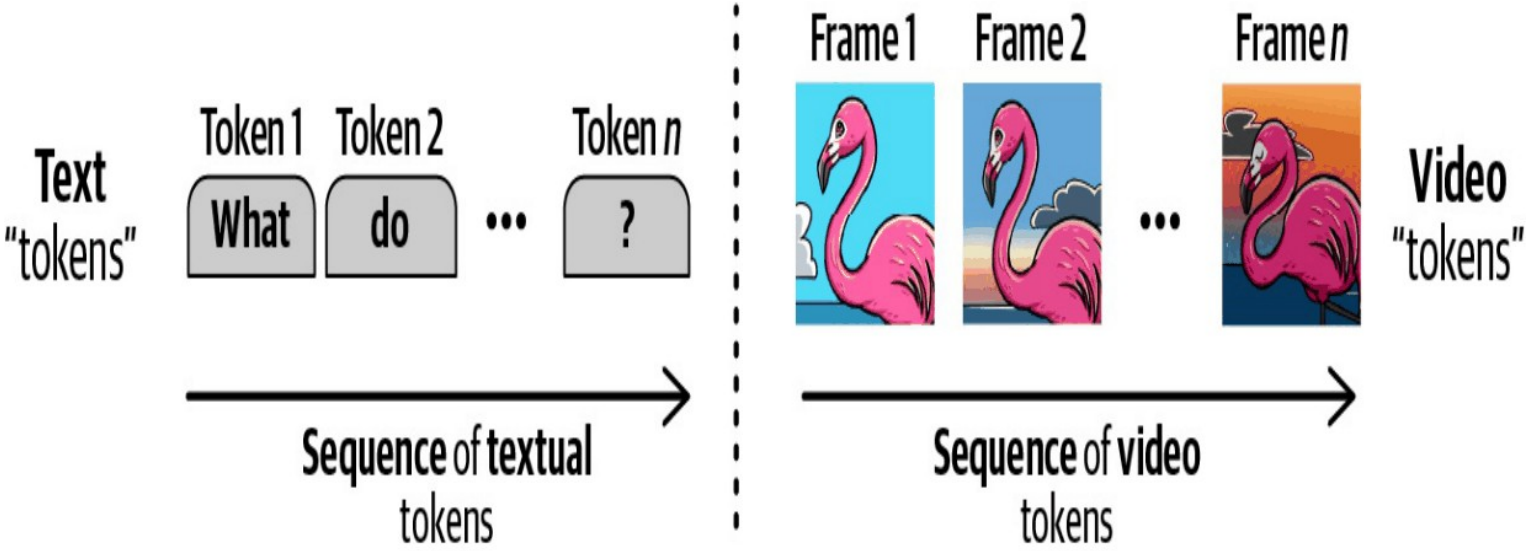
Less machinery

Fewer separate models to train, and the LLM does the temporal reasoning.



Grootendorst & Alammar (2026). Fig. 9-32, PDF p. 466

Video as a sequence, like text



Two sequences
Text is a sequence of subwords (tokens). Video is a sequence of frames (video tokens).

Same nature
Both have a time-based sequential nature, so the same architecture can process both.

In practice
Sample frames, create embeddings for each image, and pass them as a sequence to the LLM.

Grootendorst & Alammar (2026). Fig. 9-33, PDF p. 467

Videos as sequences of image embeddings

The connector's role

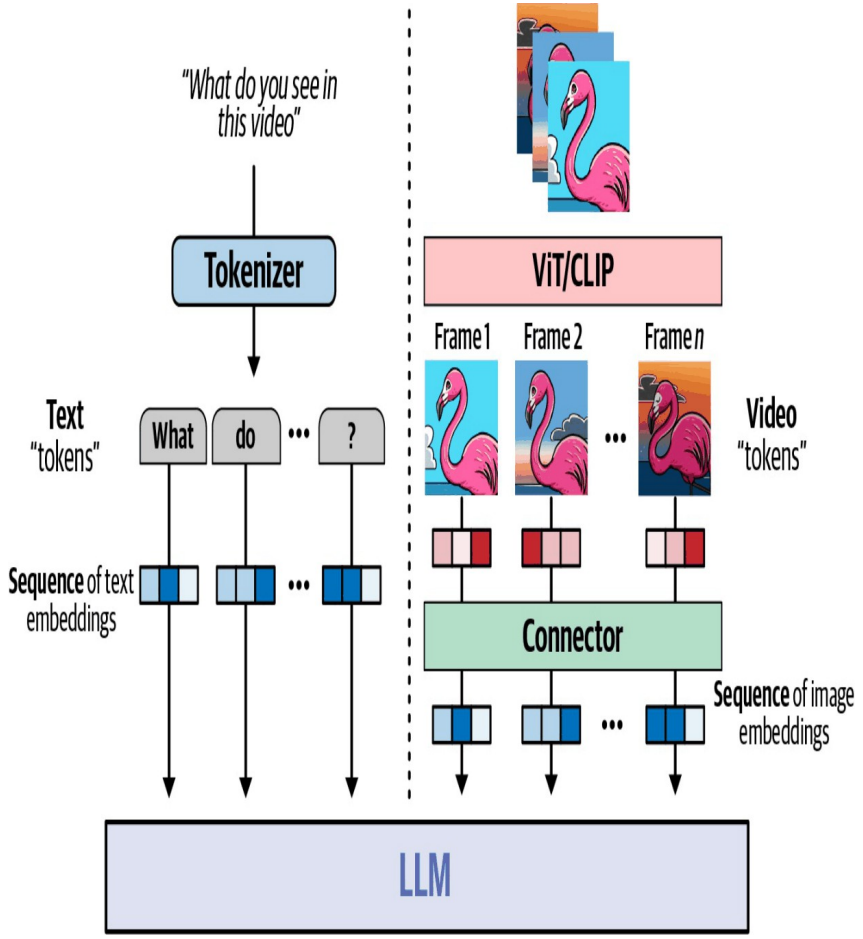
The connector can add temporal data or simply convert the image embeddings to the type of embeddings the LLM expects.

Like text

Videos are processed as sequences of image embeddings, much like text is processed as a sequence of text embeddings.

Token granularity

Video tokens can be formed from the entire image or from individual image tokens.



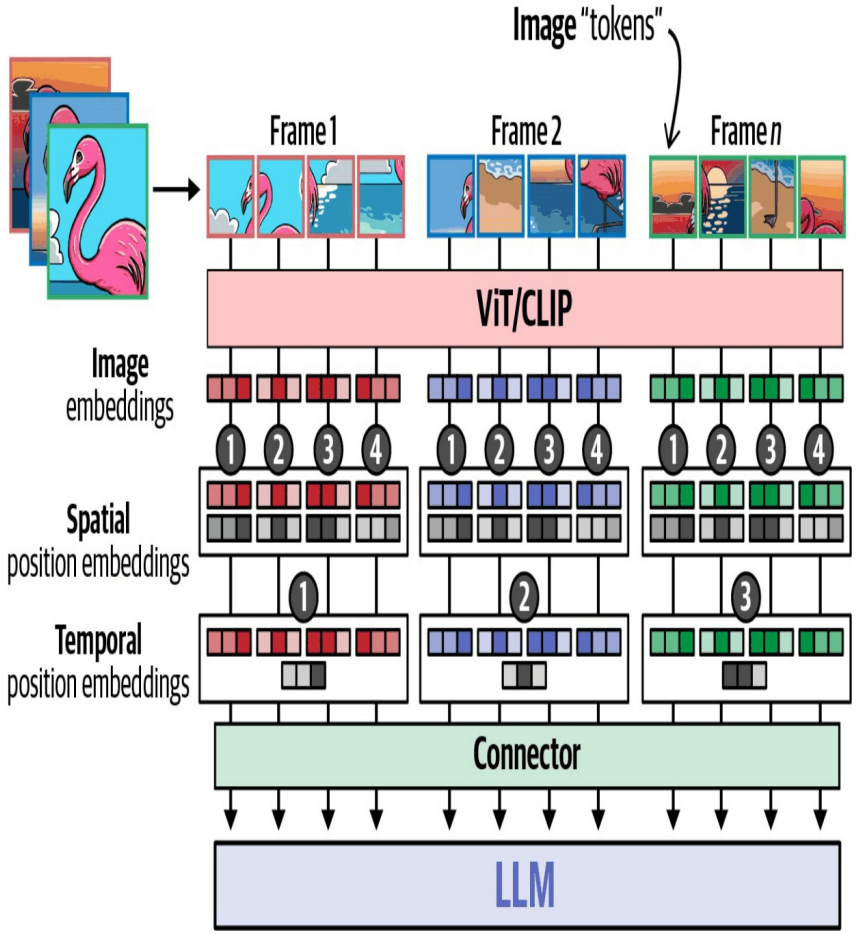
Grootendorst & Alammar (2026). Fig. 9-34, PDF p. 468

VideoMamba: positional embeddings for time and space

Enhance embeddings
VideoMamba uses positional embeddings to enhance the image embeddings with spatial and temporal information.

Spatial first
It first adds spatial information: to which part of the frame does an embedding belong?

Then temporal
Then temporal information: in which order are the frames?



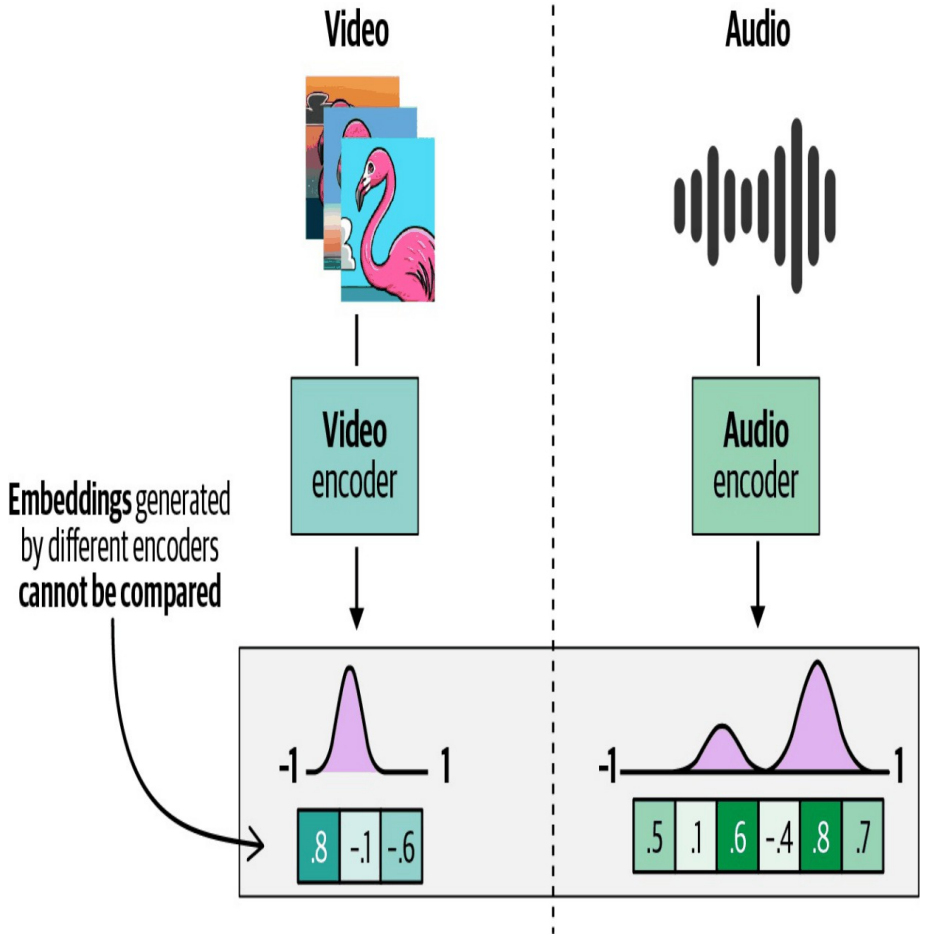
Grootendorst & Alammar (2026). Fig. 9-35, PDF p. 469

The many-modality problem

Isolated encoders
In the earlier examples, modalities are encoded alone or in pairs. That needs a different model for each modality.

Not comparable
Embeddings from different models cannot be compared. They have different value distributions and often different dimensions.

A single space
A better aim is one embedding space that can hold many modalities.



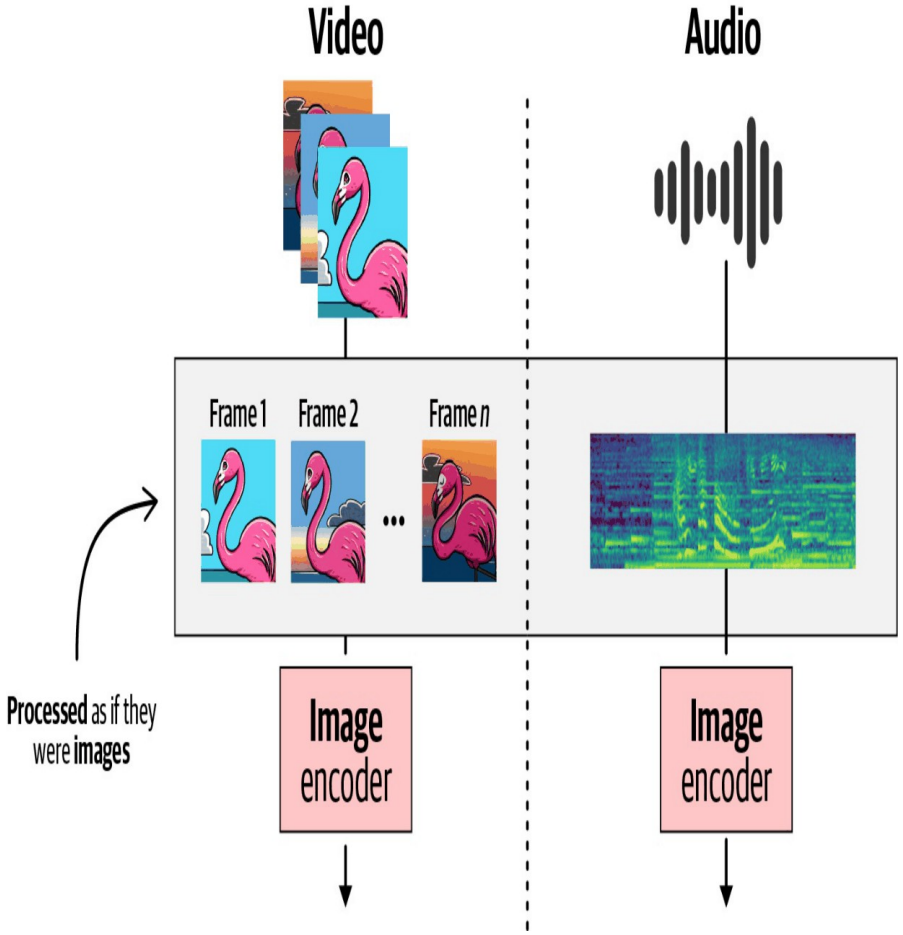
Grootendorst & Alammar (2026). Fig. 9-36, PDF p. 470

ImageBind: an image binds the modalities

One embedding space
ImageBind combines contrastive learning with an idea: a single image can bind different modalities. A rainy beach image can evoke waves, mist, and a quiet mood.

Through images
Video and audio encoders often use image encoders as their main encoder, so they produce the same type of embeddings, which can be compared.

Always paired with vision
Contrastive learning trains on pairs, and in ImageBind one member of each pair is always visual.



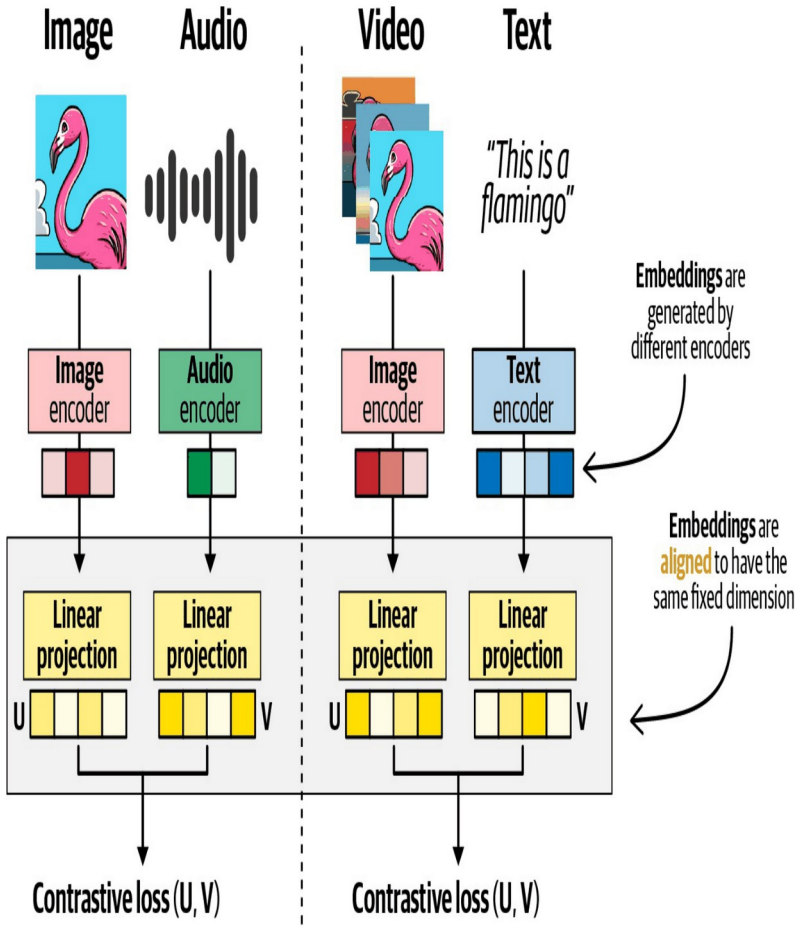
Grootendorst & Alammar (2026). Fig. 9-37, PDF p. 471

How ImageBind is trained

A linear projection
For each pair of modalities, a linear projection aligns the embeddings of the pair.

Trained pairs
Image with text, image with depth sensor data, image with thermal data, video with IMU (accelerometer) data, and video with audio.

Connected through images
All the modalities are connected through the image, as the figure shows.



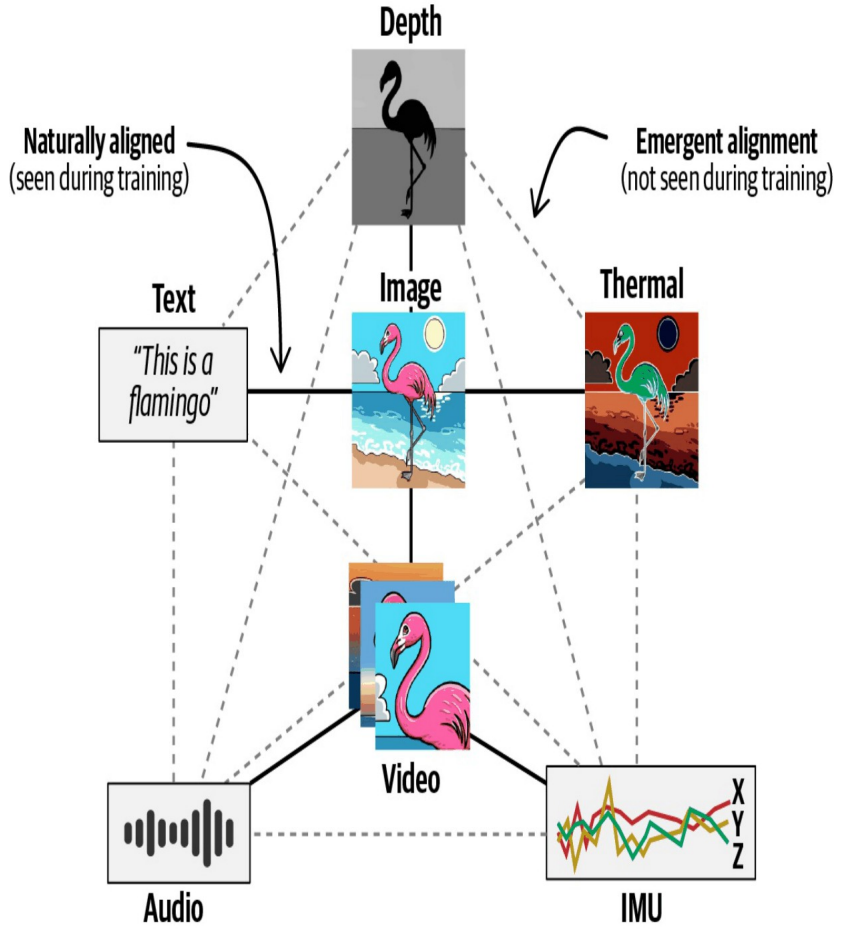
Grootendorst & Alammar (2026). Fig. 9-38, PDF p. 472

Emergent alignment of unseen pairs

Unseen pairs
Many pairs of modalities were never seen together during training, for example audio and text.

Still aligned
Even so, their embeddings were aligned in the embedding space.

The binding factor
This emergent alignment shows how strong the binding role of images can be.



Grootendorst & Alammar (2026). Fig. 9-39, PDF p. 473

Connecting modalities to the LLM

The problem

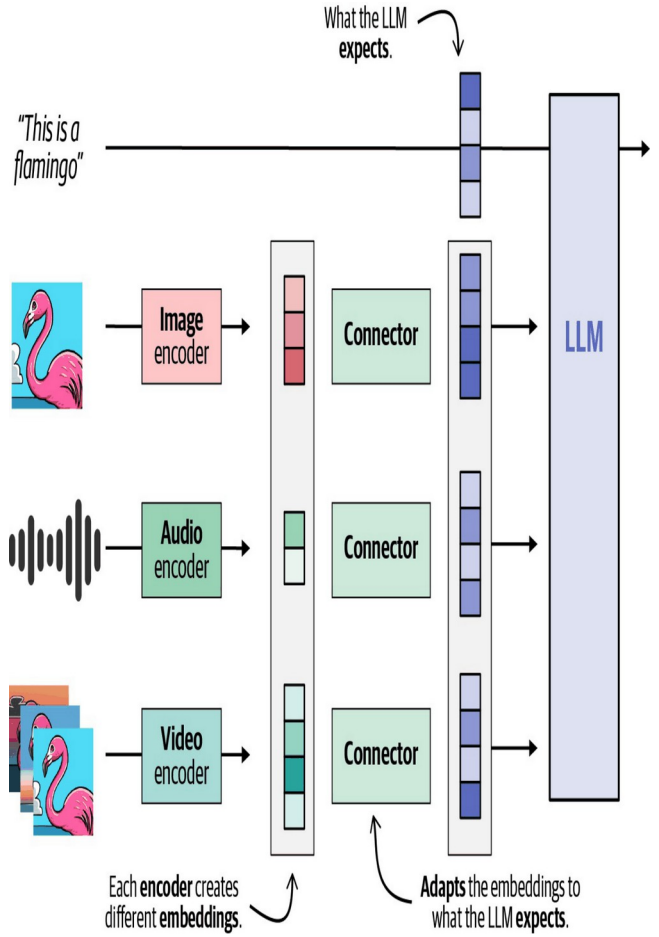
Encoders create embeddings that differ in size, dimensions, and embedding space from what the LLM expects. Positional encoding and tokenization schemas also differ.

The connector

A connector transforms them into the same type of embeddings as the LLM's token embeddings.

Per encoder

In the figure each encoder, for images, audio, and video, has a connector that adapts its output to the LLM.



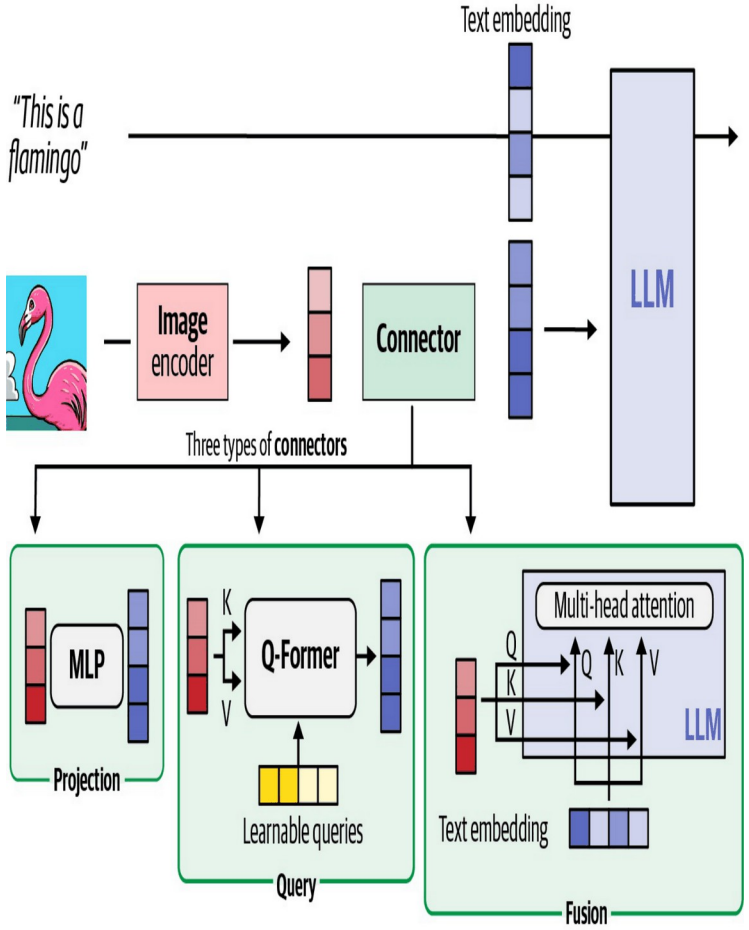
Grootendorst & Alammar (2026). Fig. 9-40, PDF p. 475

Three types of connectors

Projection-based
A Multi-Layer Perceptron (MLP) projects multi-modal embeddings to an embedding size that is compatible with the LLM.

Query-based
Groups of learnable query tokens extract information from the inputs in a query-based way.

Fusion-based
Additional cross-attention layers inside the LLM process both text and other multi-modal embeddings.



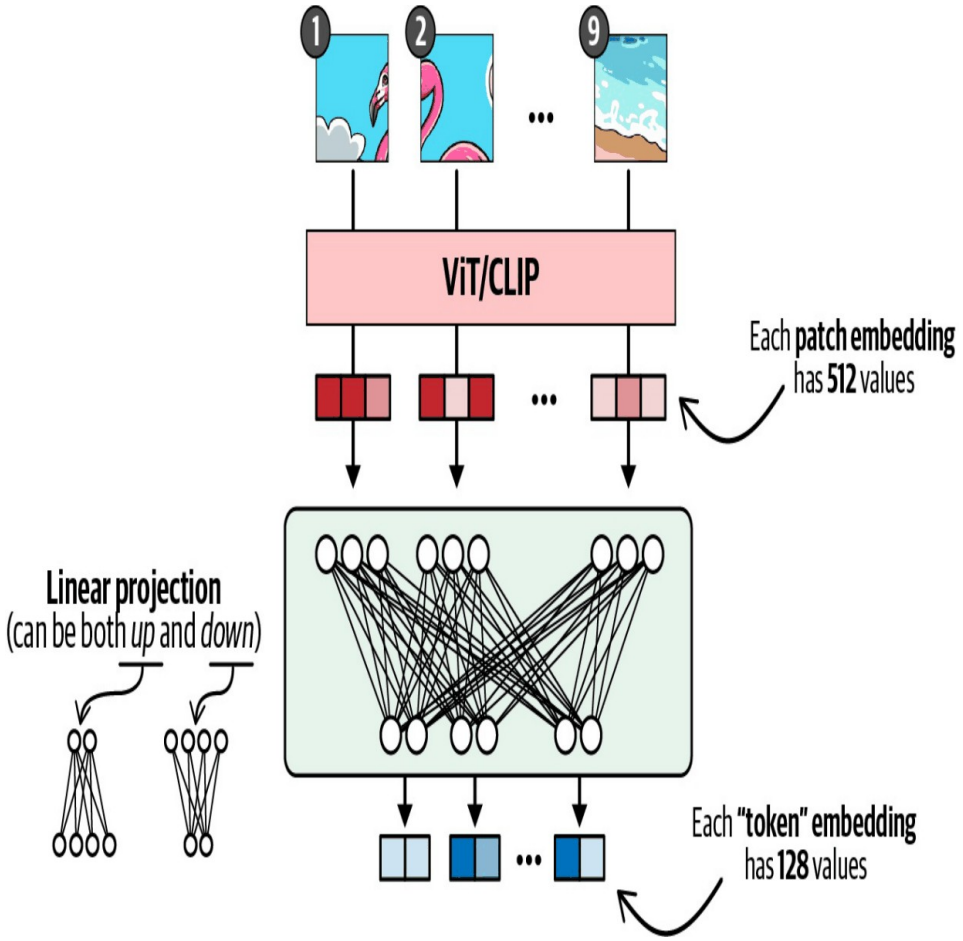
Grootendorst & Alammar (2026). Fig. 9-41, PDF p. 477

Projection-based connector

Simplest method
Align multi-modal embeddings with text embeddings by projecting them, typically with a single linear layer.

Up or down
The projection may go up (fewer values to more) or down, depending on whether the non-text embeddings are smaller or larger than the LLM's token embeddings.

Each embedding mapped
A linear projection maps each input image embedding to an output embedding of the right size.



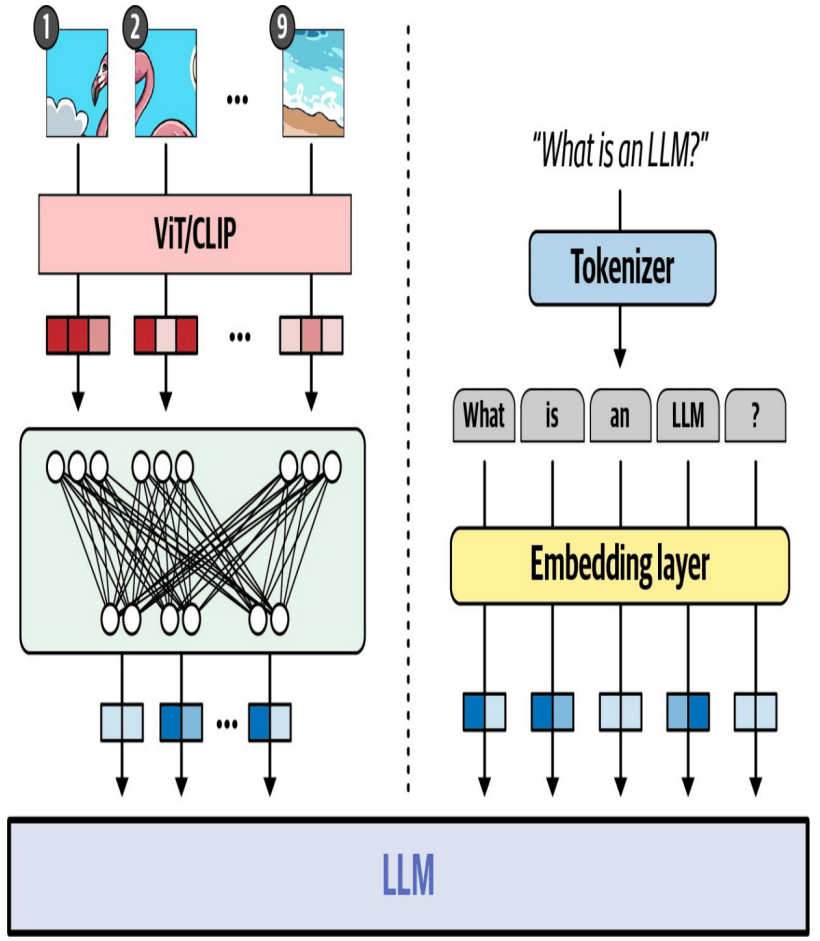
Grootendorst & Alammar (2026). Fig. 9-42, PDF p. 478

Projection in practice

Concatenate
The projected non-text embeddings are concatenated with the token embeddings of the text input.

Treated like tokens
The LLM treats every embedding as if it were a token embedding.

Matching three things
The projection matches the count, dimensions, and data distribution that the LLM expects.



Grootendorst & Alammar (2026). Fig. 9-43, PDF p. 479

LLaVA: a two-step recipe

The model

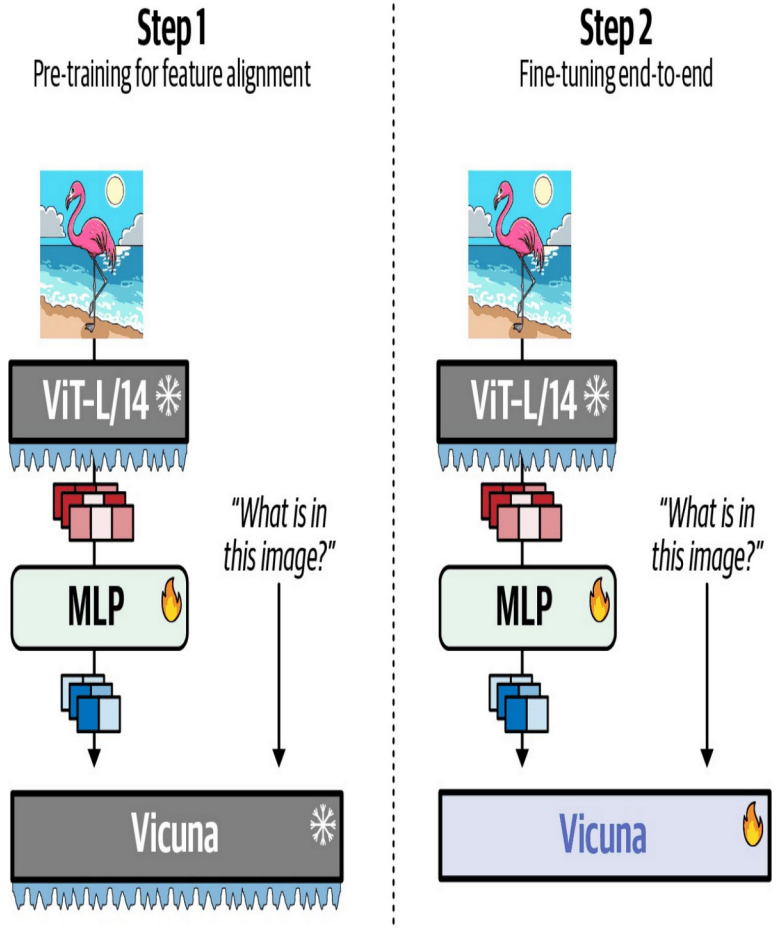
Large Language and Vision Assistant (LLaVA) makes Vicuna, a text-only Llama 2 variant, multi-modal. Vicuna was chosen in 2023 for its instruction following.

Step 1: align features

Pre-training for feature alignment. Only the projection layer is trained. The ViT and Vicuna are frozen.

Step 2: fine-tune end to end

The projection layer and the LLM are trained. Only the ViT weights stay frozen.



Grootendorst & Alammar (2026). Fig. 9-44, PDF p. 481

Query-based connector: the Q-Former

Extract what matters

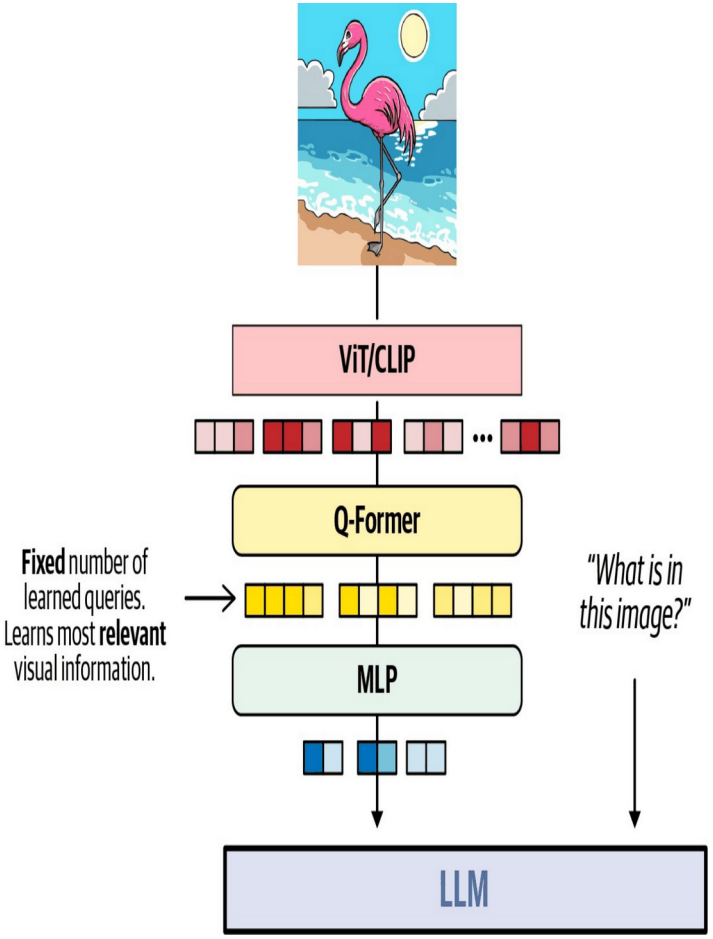
A query-based connector extracts relevant features from non-text inputs before the LLM. The learnable queries give it the name Q-Former.

Fixed number

Instead of projecting all ViT features, the Q-Former makes a fixed number of features for the LLM, so it learns only the most relevant visual information.

Then a projection

The compressed features go to a projection connector before the LLM, as the figure shows.



Grootendorst & Alammar (2026). Fig. 9-45, PDF p. 483

The Q-Former's two modes

Learnable queries

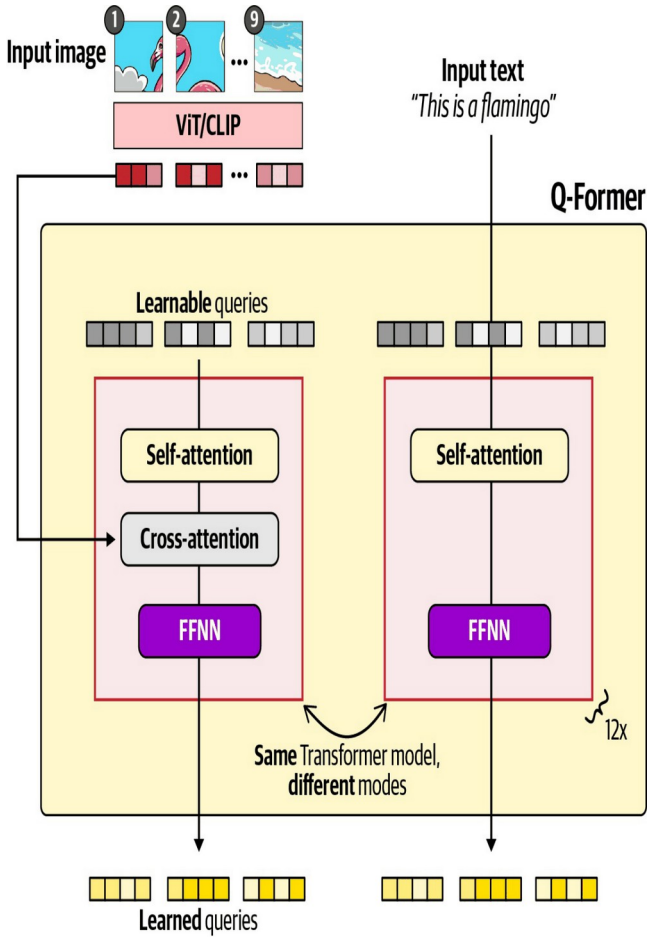
During training the Transformer receives learnable queries, embeddings that get updated with relevant visual and textual information.

Two modes

The same Transformer works in one mode with cross-attention to process images (left) and one without cross-attention for text (right).

Compression

The output is the learned queries, which hold the image embeddings' most relevant information, a compression into fewer vectors.



Grootendorst & Alammar (2026). Fig. 9-46, PDF p. 485

Training the Q-Former in two stages

Stage 1: three tasks

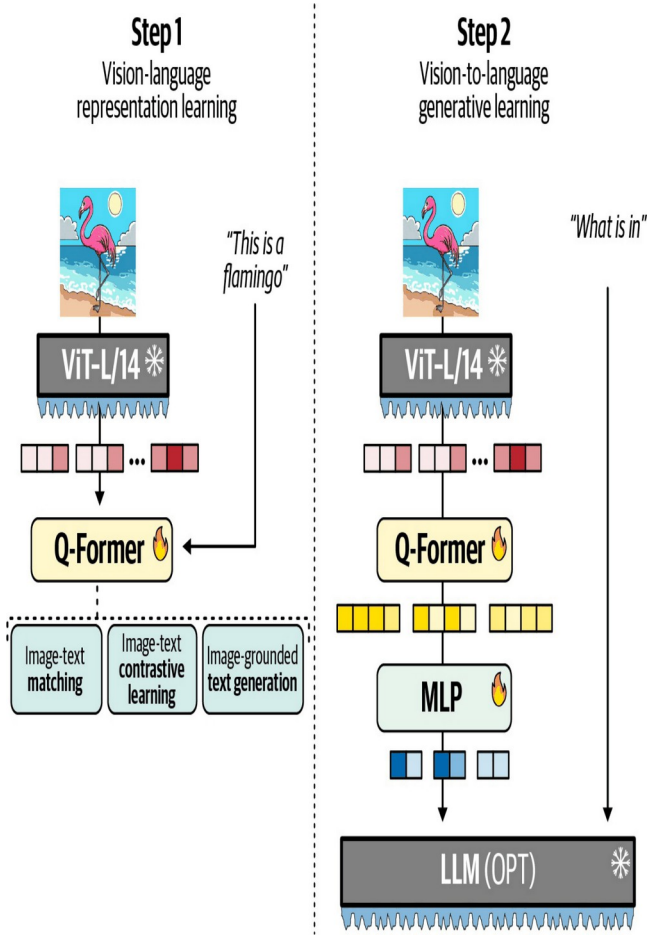
Image-text contrastive learning aligns representations.
Image-grounded text generation generates text from images.
Image-text matching predicts whether a pair matches.

Stage 2: add an LLM

The trained Q-Former is connected to an LLM (OPT). The outputs are projected through an MLP to the LLM's input size.

What is trained

Only the Q-Former and the MLP are trained. The LLM and the image encoder are frozen. Qwen-VL used this technique.



Grootendorst & Alammar (2026). Fig. 9-47, PDF p. 487

Fusion-based connector: Flamingo

Change the LLM

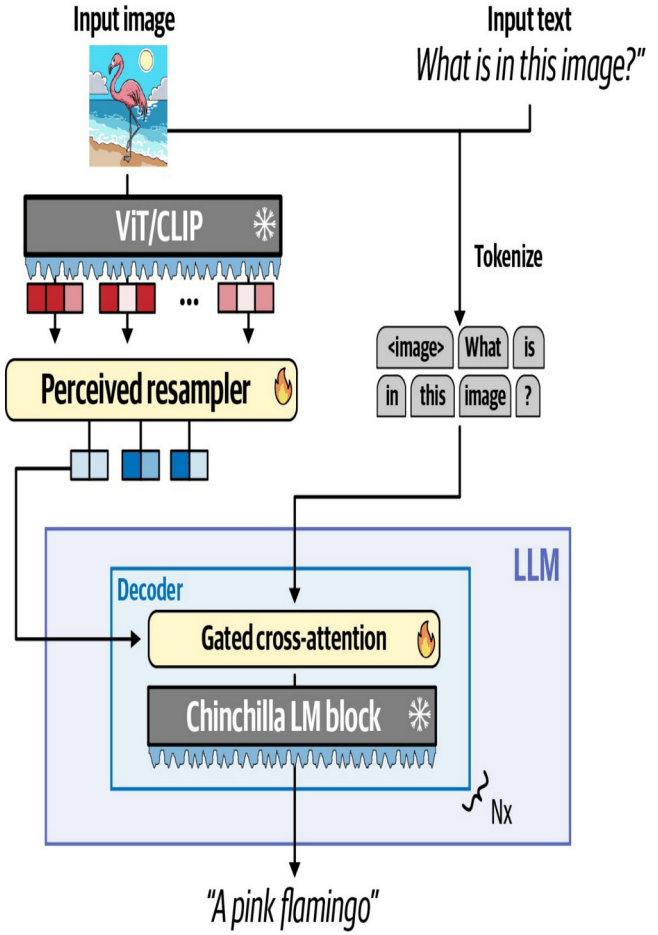
Unlike the previous two, a fusion-based connector adds modules to the LLM's architecture, for deep interaction between text and non-text features.

Flamingo

One of the first. It uses a pre-trained LLM and adds trainable cross-attention layers that process the input images.

The path

The vision encoder's output goes through a perceiver resampler, which maps it to a fixed sequence size, and then to the added cross-attention layers.



Grootendorst & Alammar (2026). Fig. 9-48, PDF p. 489

The perceiver resampler

What is trained

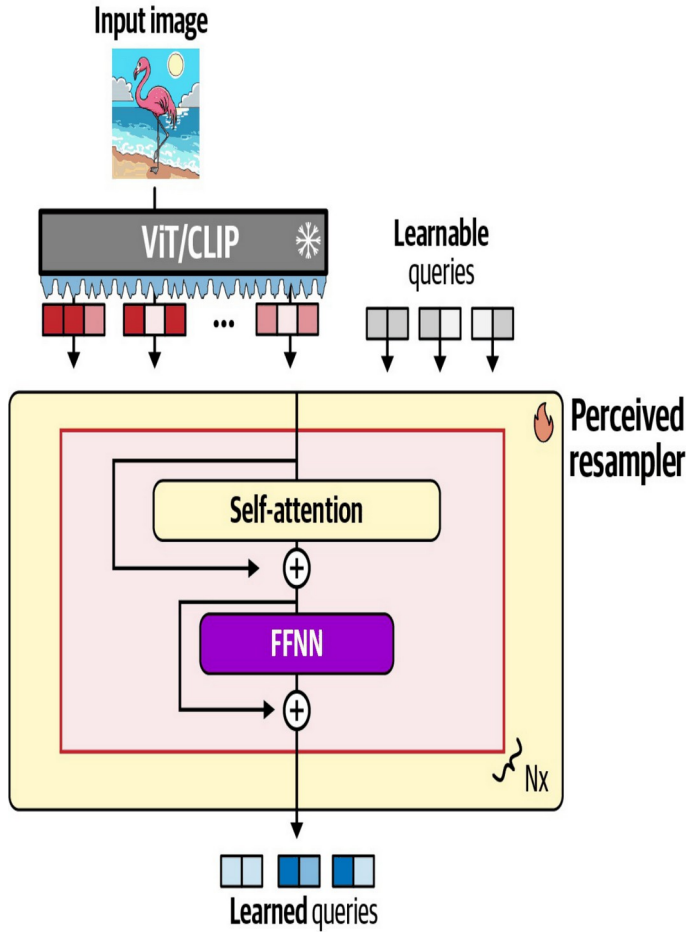
Only the added cross-attention and the perceiver resampler are trained. The pre-trained vision encoder and the LLM are frozen.

The goal

Extract the most important vision features and output them in a fixed size, like the Q-Former.

Reduced computation

The learned queries have the shape the LLM expects. A fixed size reduces computation by focusing on the most important visual features.



Grootendorst & Alammar (2026). Fig. 9-49, PDF p. 491

Gated cross-attention

Attend to both

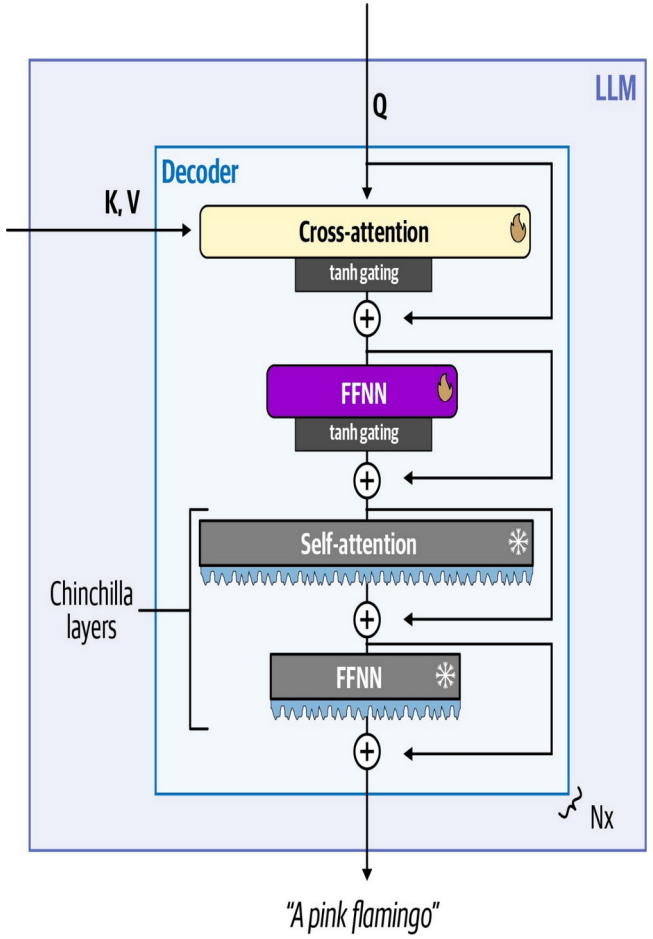
The resampler's output goes to cross-attention blocks interleaved with the LLM's regular blocks, so the LLM attends to both image and text features.

The gating trick

Layer outputs are multiplied by tanh of a learnable scalar alpha, initialized to 0. Since tanh(0) is 0, the layers initially contribute nothing.

Stability

The model initially behaves like the original LLM. As training goes on, alpha learns to open the gate, which helps stability and final performance.



Grootendorst & Alammar (2026). Fig. 9-50, PDF p. 493

Advantages and disadvantages of connectors

Projection-based
Advantages: simple and lightweight, low latency, few parameters. Disadvantages: can produce a large number of tokens that fill the context window, and limited expressive power because the model is small and decoupled from the LLM.

Query-based
Advantages: a fixed number of queries scales to large inputs such as videos, with more expressive power than projection. Disadvantages: complex training, and more latency than projection because of larger models.

Fusion-based
Advantages: integration with the LLM lets it see multi-modal features at every layer and not only as a prefix, with the most expressive power. Disadvantages: changes the LLM's architecture and needs extensive fine-tuning.

Grootendorst & Alammar (2026). Table 9-1, PDF p. 494

Choosing a connector along three axes

Projection: simplicity and efficiency

Used for simplicity and computational efficiency, since it is lightweight and easy to train. The input is not compressed, so it may quickly fill the context window.

Query: efficiency and scale

Used for efficiency and scaling, because the input length stays constant regardless of image or video resolution. It may skip over details that projection keeps.

Fusion: performance

Tends to perform best, because the multi-modal input is processed throughout the model and not only as token embeddings. It is the most complex and most expensive to train.

TinyAgent: vision tokens in Gemma 4 E4B

A model with two encoders

The model used throughout the book has both a vision encoder for images and an audio encoder for sound.

Image tokens

The vision encoder produces embeddings, also called vision or image tokens. In the chat template, special tokens mark the start and end of a run of image tokens.

Resolution

Gemma 4 E4B uses a maximum of 70, 140, 280, 560, or 1,120 vision tokens. More tokens mean a higher resolution is processed.

Handled by the engine

The inference engine builds this template. The agent only sends a message whose content has an image field and a text field.

Grootendorst & Alammar (2026). Code, PDF pp. 495–496

Adding images to TinyAgent's memory

Messages carry the image

Messages are both the memory of the model and the way we communicate with it, as Chapter 4 showed. So images are enabled in the Memory class.

A small change

A multi-modal memory adds an image to the message content if one is given, in addition to the text.

URL or base64

The image can be a link or a base64-encoded string. Some inference engines fetch the link themselves, and others accept only base64.

The run function

The agent's run function gets an optional image parameter and passes it to memory. Nothing else changes.

Does the agent see? Without and with the image

The question

The agent is asked which animal is on the cover of the book Hands-On Large Language Models, first without an image.

Without the image

It says it cannot view the cover or know the edition, and asks for an image.

With the image

Given the cover as base64 data, the agent answers that the animal is a kangaroo. That is correct.

Encoder, connector, and LLM at a glance

Image

Encoder: ViT or CLIP, which cuts the image into patches. Connector: often a projection. Example: LLaVA.

Audio

Encoder: Whisper's encoder or CLAP's audio encoder, built on spectrograms or strided waveforms. Connected like images.

Video

Encoder: an image encoder applied to sampled frames, perhaps with a temporal step. The connector or the LLM handles order.

Many modalities

ImageBind aligns images, text, audio, depth, thermal, and motion data in one space by binding each to images.

Choosing a connector for a project

A quick prototype

A small team adds image understanding to an existing model. Projection fits: simple and cheap, with the context cost to watch.

Long videos

Many frames would fill the context. Query-based keeps the number of embeddings fixed.

Maximum quality

A lab with compute and control of the model might choose fusion for the best use of visual detail.

Fixed model

If you use a hosted model, you do not choose. You can only choose which modalities to send and how many tokens they use.

Teaching addition. Concepts: Ch. 9, PDF pp. 476–495

Using multi-modal agents responsibly

More sensitive inputs

Images, audio, and video can contain faces, voices, and locations. What is sent to a model, and where it is processed, needs a decision.

Cost and context

Each image or video frame consumes tokens. Resolution and frame sampling are cost and capacity decisions.

Check what it sees

A model may miss or misread details. Evaluation sets for multi-modal tasks should include images and audio like the real ones.

Record the inputs

For an agent that acts on what it sees, keeping the inputs together with the trajectory supports review.

Teaching addition. Concepts: Ch. 9. Cross-reference: governance half of the course

Activity: trace an image through the pipeline

The scenario

An agent receives a photograph of a whiteboard and the question: what are the action items? Draw the pipeline from input to answer.

The steps

Label the encoder, the output embeddings, the connector, and the LLM. Say what is a vector at each stage and what is text.

The variations

Replace the photo with a recorded meeting. Which encoder would you use for the audio? What changes if the meeting were a video?

Teaching addition. Concepts: Ch. 9, PDF pp. 429–498

Learning check with worked answers

1. Why must modalities become embeddings?
The LLM takes embeddings as input. An encoder turns an image, sound, or video into embeddings, so they resemble token embeddings.

2. Why is CLIP a popular encoder?
It is trained contrastively on image and text pairs, so its image embeddings have already been exposed to the textual modality.

3. Projection versus query?
Projection maps every embedding and can fill the context. A query-based connector compresses to a fixed number of embeddings but is more complex to train.

4. What does gating do in Flamingo?
Tanh gating starts at zero, so the model behaves like the original LLM at first. The gate opens as training goes on, which improves stability.

Chapter summary

Encoding modalities

Images (ViT and CLIP), audio (wav2vec 2.0, Whisper, CLAP), and video (ViViT, TimeSformer, and just CLIP) are mostly encoded by Transformers. ImageBind processes many modalities at once.

Connecting to LLMs

Projection-based connectors use an MLP. Query-based connectors use a Transformer plus an MLP. Fusion-based connectors add cross-attention blocks to the pre-trained LLM that can be trained.

TinyAgent

Giving TinyAgent image understanding required only small changes to memory and to the run function. The next chapter covers coding agents.

Core distinctions and source guide

Conceptual distinctions

A multi-modal LLM has an encoder per modality, a connector that makes the embeddings usable by the LLM, and the LLM itself. Encoders turn input into embeddings. Connectors project, query, or fuse those embeddings into the language model.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 9. Selected connections: Chapters 2, 4, 5, and 8.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF